

实战章节：在Linux上部署各类软件

前言

为什么学习各类软件在Linux上的部署

在前面，我们学习了许多的Linux命令和高级技巧，这些知识点比较零散，同学们跟着课程的内容进行练习虽然可以基础掌握这些命令和技巧的使用，但是并没有一些具体的实操能够串联起来这些知识点。

所以，现在我们设计了各类软件在Linux上部署安装的实战章节，可以让同学们：

- 对前面学习的各类操作命令进行复习和练习，从而深度掌握它们
- 本章节中演示部署的软件，包含了IT行业各类岗位中所必须使用的，如：Java 后台、大数据开发、运维开发、测试、AI等。无论学习Linux后从事什么岗位，这些内容都会给你带来帮助

对于零基础学员，实战课程中所讲解的软件大概率多数大家并不了解。

所以，课程仅涉及到安装部署，不对软件的使用做详细说明。

同学们在这个过程中，可能会遇到各种各样的错误，**不要怕**，解决它，将会给你带来极大的提升。

学习目标

对于本部分的内容学习，我们设计两个目标：

- 对于零基础或未从业的学员，不要求深入理解所安装部署的软件是什么，仅仅能够跟随课程成功的将其部署安装并运行成功即可
在这个过程中，主要锻炼大家对Linux操作系统的熟练度，此乃零基础未从业学员的第一学习目标
- 对于有基础或已从业的学员，本章节讲解的软件涵盖了大多数IT从业者所能接触到的，特别是大数据开发、后端开发两个主流方向，可以作为参考资料，以便在工作中有所帮助。

本章节内的各类软件安装，**不强制要求全部学习**

1. 零基础学员，建议全部学习，作为前面学习内容的总结和实战

2. IT从业者、有经验学员，可以按需选择，选择工作中需要用到的进行学习

章节内包含的软件并非100%涵盖了IT开发领域中所需要的内容。

如果您对某些软件的安装有强烈需求，且课程中没有提供教程，可以私信B站："黑马大数据曹老师"，老师会酌情根据时间安排补充上去哦。

为什么不使用PPT而是使用文档进行授课

从现在开始，将要进入到Linux的实操阶段，在这个阶段我们将会涉及到非常多的软件部署等操作，涉及到：

- 各类命令的使用
- 各种过程的结果
- 复杂的流程步骤
- 等

这些信息的展示，并不适合于使用PPT作为载体进行授课，所以从现在开始我们将使用操作文档的模式为大家进行讲解。

使用文档模式还有一个好处，除了学习视频以外，拿到课程中使用的操作文档，同样可以作为重要的参考手册进行使用。

一举两得

前置要求

1. 实战章节要求同学们 **务必全部学习前面的知识点**，即：初识Linux、Linux基础命令、Linux权限管理、Linux高阶技巧这4个章节，请勿跳过前面的章节学习实战章节。
2. 实战章节中会开启多台虚拟机，请尽量确保电脑的内存存在：8GB（包含8GB）以上。如内存不足可以扩充内存条或购买阿里云、UCloud等云服务器临时使用（1个月多台低配服务器几十块左右）

对于云平台上购买服务器，可以参阅最后的章节（云服务）

注意

下面全部的软件安装的相关流程，90%都是取自软件自身的官方网站。

一个合格的程序员要有良好的信息收集能力哦

MySQL数据库管理系统安装部署【简单】

简介

MySQL数据库管理系统（后续简称MySQL），是一款知名的数据库系统，其特点是：轻量、简单、功能丰富。

MySQL数据库可谓是软件行业的明星产品，无论是后端开发、大数据、AI、运维、测试等各类岗位，基本上都会和MySQL打交道。

让我们从MySQL开始，进行实战的Linux软件安装部署。

本次课程分为2个版本进行安装：

- MySQL 5.7版本安装
- MySQL 8.x版本安装

由于MySQL5.x和8.x各自有许多使用者，所以这两个版本我们都演示安装一遍

注意

MySQL的安装过程中，除了会使用Linux命令外，还会使用到少量的数据库专用的：SQL语句

对于SQL语句我们并未涉及，所以可以跟随教程的内容，复制粘贴即可

如对MySQL感兴趣，可以学习Bilibili黑马程序员视频：[2022新版黑马程序员MySQL知识精讲+mysql实战案例_零基础mysql数据库入门到高级全套教程](#)

如有时间，建议可以在学习完Linux系统之后，学习一下MySQL数据库

无论从事什么方面的开发，Java后端、大数据、AI、前端、Linux运维等，都会要求掌握MySQL数据库的

可以说，MySQL是IT开发从业者必备的技能了。

MySQL5.7版本在CentOS系统安装

注意：安装操作需要root权限

MySQL的安装我们可以通过前面学习的yum命令进行。

安装

1. 配置yum仓库

```
1 # 更新密钥
2 rpm --import https://repo.mysql.com/RPM-GPG-KEY-
  mysql-2022
3
4 # 安装Mysql yum库
5 rpm -Uvh http://repo.mysql.com//mysql57-community-
  release-el7-7.noarch.rpm
```

```
[root@centos ~]# rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2022
[root@centos ~]# rpm -Uvh http://repo.mysql.com/mysql57-community-release-el7-7.noarch.rpm
Retrieving http://repo.mysql.com/mysql57-community-release-el7-7.noarch.rpm
warning: /var/tmp/rpm-tmp.AEfpum: Header V3 DSA/SHA1 Signature, key ID 5072e1f5: NOKEY
Preparing... ##### [100%]
Updating / installing...
 1:mysql57-community-release-el7-7 ##### [100%]
```

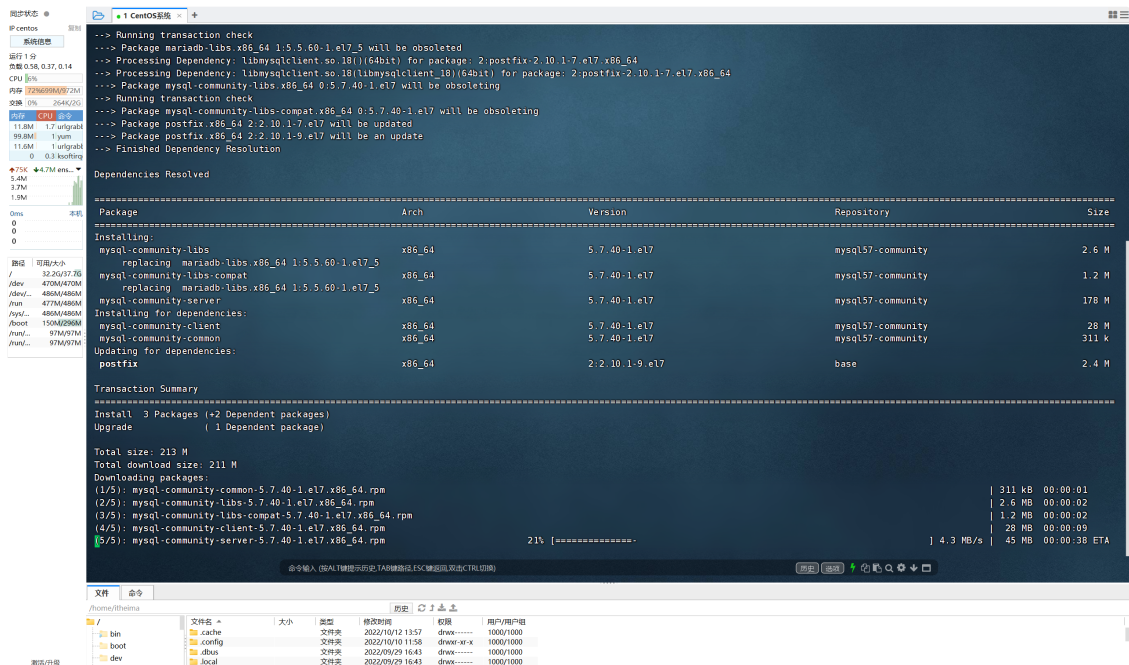
由于MySQL并不在CentOS的官方仓库中，所以我们通过上述rpm命令：

- 导入MySQL仓库的密钥

○ 配置MySQLQ的yum仓库

2. 使用yum安装MySQL

```
1 # yum安装Mysql
2 yum -y install mysql-community-server
```



3. 安装完成后，启动MySQL并配置开机自启动

```
1 systemctl start mysqld # 启动
2 systemctl enable mysqld # 开机自启
```

MySQL安装完成后，会自动配置为名称叫做：**mysqld**的服务，可以被systemctl所管理

4. 检查MySQL的运行状态

```
1 systemctl status mysqld
```

```
[root@centos ~]# systemctl status mysqld
● mysqld.service - MySQL Server
   Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; vendor preset: disabled)
   Active: active (running) since Wed 2022-10-12 18:26:57 CST; 4s ago
     Docs: man:mysqld(8)
           http://dev.mysql.com/doc/refman/en/using-systemd.html
   Main PID: 13560 (mysqld)
    CGroup: /system.slice/mysqld.service
            └─13560 /usr/sbin/mysqld --daemonize --pid-file=/var/run/mysqld/mysqld.pid

Oct 12 18:26:53 centos systemd[1]: Starting MySQL Server...
Oct 12 18:26:57 centos systemd[1]: Started MySQL Server.
```

配置

主要配置管理员用户root的密码以及配置允许远程登录的权限。

1. 获取MySQL的初始密码

```
1 # 通过grep命令, 在/var/log/mysqld.log文件中, 过滤  
   temporary password关键字, 得到初始密码  
2 grep 'temporary password' /var/log/mysqld.log
```

```
[root@centos ~]# grep 'temporary password' /var/log/mysqld.log  
2022-10-12T10:26:54.826339Z 1 [Note] A temporary password is generated for root@localhost: iT?atBiXsloe
```

2. 登陆MySQL数据库系统

```
1 # 执行  
2 mysql -uroot -p  
3 # 解释  
4 # -u, 登陆的用户, MySQL数据库的管理员用户同Linux一样, 是  
   root  
5 # -p, 表示使用密码登陆  
6  
7 # 执行完毕后输入刚刚得到的初始密码, 即可进入MySQL数据库
```

```
[root@centos ~]# mysql -uroot -p  
Enter password:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 2  
Server version: 5.7.40  
  
Copyright (c) 2000, 2022, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
mysql>
```

3. 修改root用户密码

```
1 # 在MySQL控制台内执行  
2 ALTER USER 'root'@'localhost' IDENTIFIED BY '密码';  
   -- 密码需要符合: 大于8位, 有大写字母, 有特殊符号, 不能是连续的  
   简单语句如123, abc
```

4. [扩展], 配置root的简单密码

我们可以给root设置简单密码，如123456。

请注意，此配置仅仅是用于测试环境或学习环境的MySQL，如果是正式使用，请勿设置简单密码

```
1 # 如果你想设置简单密码，需要降低MySQL的密码安全级别
2 set global validate_password_policy=LOW; # 密码安全级别低
3 set global validate_password_length=4; # 密码长度最低4位即可
4
5 # 然后就可以用简单密码了（课程中使用简单密码，为了方便，生产中不要这样）
6 ALTER USER 'root'@'localhost' IDENTIFIED BY '简单密码';
```

5. [扩展]，配置root运行远程登录

默认情况下，root用户是不运行远程登录的，只允许在MySQL所在的Linux服务器登陆MySQL系统

请注意，允许root远程登录会带来安全风险

```
1 # 授权root远程登录
2 grant all privileges on *.* to root@"IP地址"
   identified by '密码' with grant option;
3 # IP地址即允许登陆的IP地址，也可以填写%，表示允许任何地址
4 # 密码表示给远程登录独立设置密码，和本地登陆的密码可以不同
5
6 # 刷新权限，生效
7 flush privileges;
```

6. 退出MySQL控制台页面

```
1 # 退出命令
2 exit
3
4 # 或者通过快捷键退出：ctrl + d
```

7. 检查端口

MySQL默认绑定了3306端口，可以通过端口占用检查MySQL的网络状态

```
1 | netstat -anp | grep 3306
```

```
[root@centos ~]# netstat -anp | grep 3306
tcp6      0      0 :::3306          :::*              LISTEN      13560/mysqld
[root@centos ~]# ps -ef | grep 13560
mysql     13560      1  0 18:26 ?        00:00:00 /usr/sbin/mysqld --daemonize --pid-file=/var/run/mysqld/mysqld.pid
root      32421   8466  0 18:37 pts/0    00:00:00 grep --color=auto 13560
```

至此，MySQL就安装完成并可用了，请妥善保存好MySQL的root密码。

MySQL8.0版本在CentOS系统安装

注意：安装操作需要root权限

安装

1. 配置yum仓库

```
1 # 更新密钥
2 rpm --import https://repo.mysql.com/RPM-GPG-KEY-
  mysql-2022
3
4 # 安装Mysql8.x版本 yum库
5 rpm -Uvh https://dev.mysql.com/get/mysql80-
  community-release-el7-2.noarch.rpm
```

2. 使用yum安装MySQL

```
1 # yum安装Mysql
2 yum -y install mysql-community-server
```

3. 安装完成后，启动MySQL并配置开机自启动

```
1 systemctl start mysqld      # 启动
2 systemctl enable mysqld     # 开机自启
```


MySQL安装完成后，会自动配置为名称叫做：**mysqld**的服务，可以被systemctl所管理

4. 检查MySQL的运行状态

```
1 | systemctl status mysqld
```

配置

主要修改root密码和允许root远程登录

1. 获取MySQL的初始密码

```
1 | # 通过grep命令，在/var/log/mysqld.log文件中，过滤  
    temporary password关键字，得到初始密码  
2 | grep 'temporary password' /var/log/mysqld.log
```

2. 登录MySQL数据库系统

```
1 | # 执行  
2 | mysql -uroot -p  
3 | # 解释  
4 | # -u, 登陆的用户，MySQL数据库的管理员用户同Linux一样，是  
    root  
5 | # -p, 表示使用密码登陆  
6 |  
7 | # 执行完毕后输入刚刚得到的初始密码，即可进入MySQL数据库
```

3. 修改root密码

```
1 | ALTER USER 'root'@'localhost' IDENTIFIED WITH  
    mysql_native_password BY '密码';    -- 密码需要符合：大  
    于8位，有大写字母，有特殊符号，不能是连续的简单语句如123, abc
```

4. [扩展]，配置root的简单密码

我们可以给root设置简单密码，如123456。

请注意，此配置仅仅是用于测试环境或学习环境的MySQL，如果是正式使用，请勿设置简单密码

```
1 set global validate_password.policy=0;      # 密码安全级别低
2 set global validate_password.length=4;      # 密码长度最低4位即可
```

5. 允许root远程登录，并设置远程登录密码

默认情况下，root用户是不运行远程登录的，只允许在MySQL所在的Linux服务器登陆MySQL系统

请注意，允许root远程登录会带来安全风险

```
1 # 第一次设置root远程登录，并配置远程密码使用如下SQL命令
2 create user 'root'@'%' IDENTIFIED WITH
  mysql_native_password BY '密码!'; -- 密码需要符合：大于8位，有大写字母，有特殊符号，不能是连续的简单语句如123，abc
3
4 # 后续修改密码使用如下SQL命令
5 ALTER USER 'root'@'%' IDENTIFIED WITH
  mysql_native_password BY '密码';
```

6. 退出MySQL控制台页面

```
1 # 退出命令
2 exit
3
4 # 或者通过快捷键退出：ctrl + d
```

7. 检查端口

MySQL默认绑定了3306端口，可以通过端口占用检查MySQL的网络状态

```
1 netstat -anp | grep 3306
```

```
[root@centos ~]# netstat -anp | grep 3306
tcp6      0      0 :::3306          :::*              LISTEN      14107/mysqld
tcp6      0      0 :::33060         :::*              LISTEN      14107/mysqld
tcp6      0      0 192.168.88.130:3306 192.168.88.1:5356 ESTABLISHED 14107/mysqld
[root@centos ~]# ps -ef | grep 14107
root      7477   6177   0 19:22 pts/2    00:00:00 grep --color=auto 14107
mysql     14107   1      0 18:45 ?        00:00:07 /usr/sbin/mysqld
```

至此，MySQL就安装完成并可用了，请妥善保存好MySQL的root密码。

MySQL5.7版本在Ubuntu（WSL环境）系统安装

课程中配置的WSL环境是最新的Ubuntu22.04版本，这个版本的软件商店内置的MySQL是8.0版本

所以我们需要额外的步骤才可以安装5.7版本的MySQL

安装操作需root权限，你可以：

1. 通过 `sudo su -`，切换到root用户

课程中选择这种方式操作

2. 或在每一个命令前，加上sudo，用来临时提升权限

安装

1. 下载apt仓库文件

```
1 # 下载apt仓库的安装包，Ubuntu的安装包是.deb文件
2 wget https://dev.mysql.com/get/mysql-apt-
  config_0.8.12-1_all.deb
```

```
~ > sudo wget https://dev.mysql.com/get/mysql-apt-config_0.8.12-1_all.deb
[sudo] password for caoyu:
--2022-10-16 09:40:52-- https://dev.mysql.com/get/mysql-apt-config_0.8.12-1_all.deb
Resolving dev.mysql.com (dev.mysql.com)... 23.10.6.175, 2600:140b:2:99d::2e31, 2600:140b:2:99c::2e31
Connecting to dev.mysql.com (dev.mysql.com)|23.10.6.175|:443... connected.
HTTP request sent, awaiting response... 302 Moved Temporarily
Location: https://repo.mysql.com//mysql-apt-config_0.8.12-1_all.deb [following]
--2022-10-16 09:40:52-- https://repo.mysql.com//mysql-apt-config_0.8.12-1_all.deb
Resolving repo.mysql.com (repo.mysql.com)... 23.192.208.228
Connecting to repo.mysql.com (repo.mysql.com)|23.192.208.228|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 36306 (35K) [application/x-debian-package]
Saving to: 'mysql-apt-config_0.8.12-1_all.deb'

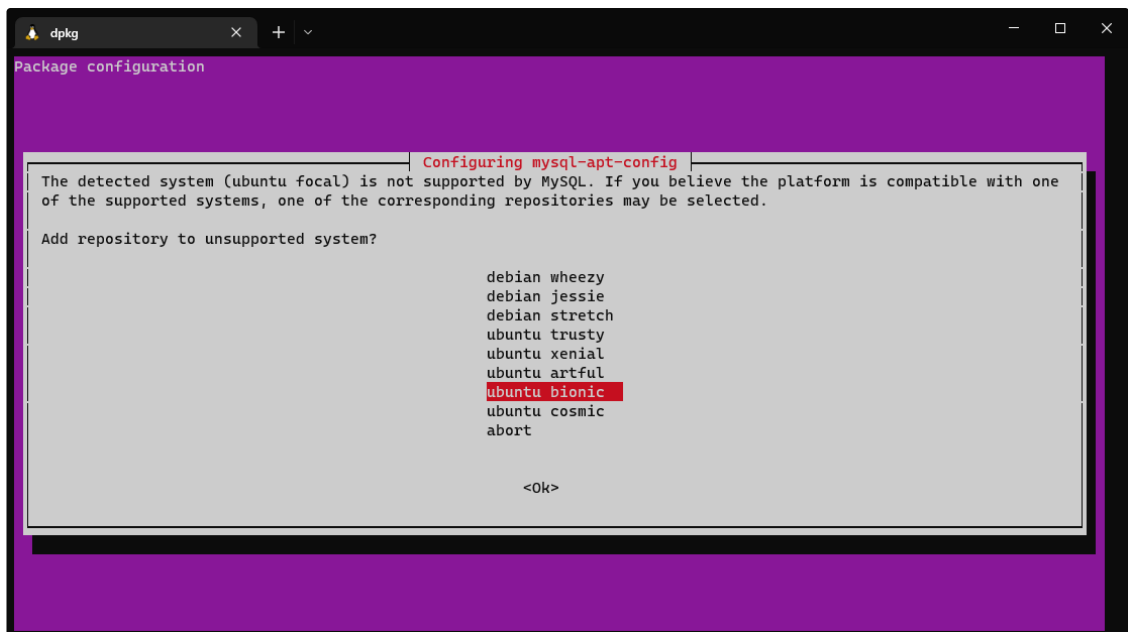
mysql-apt-config_0.8.12-1_all 100%[=====] 35.46K 172KB/s in 0.2s

2022-10-16 09:40:53 (172 KB/s) - 'mysql-apt-config_0.8.12-1_all.deb' saved [36306/36306]
```

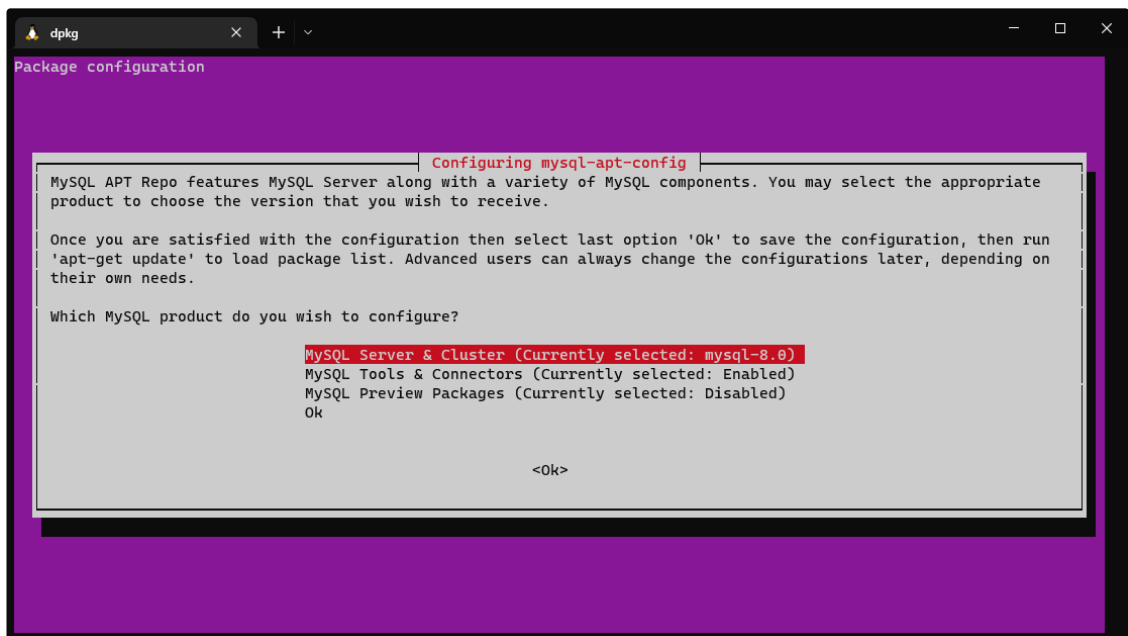
2. 配置apt仓库

- 1 # 使用dpkg命令安装仓库
- 2 dpkg -i mysql-apt-config_0.8.12-1_all.deb

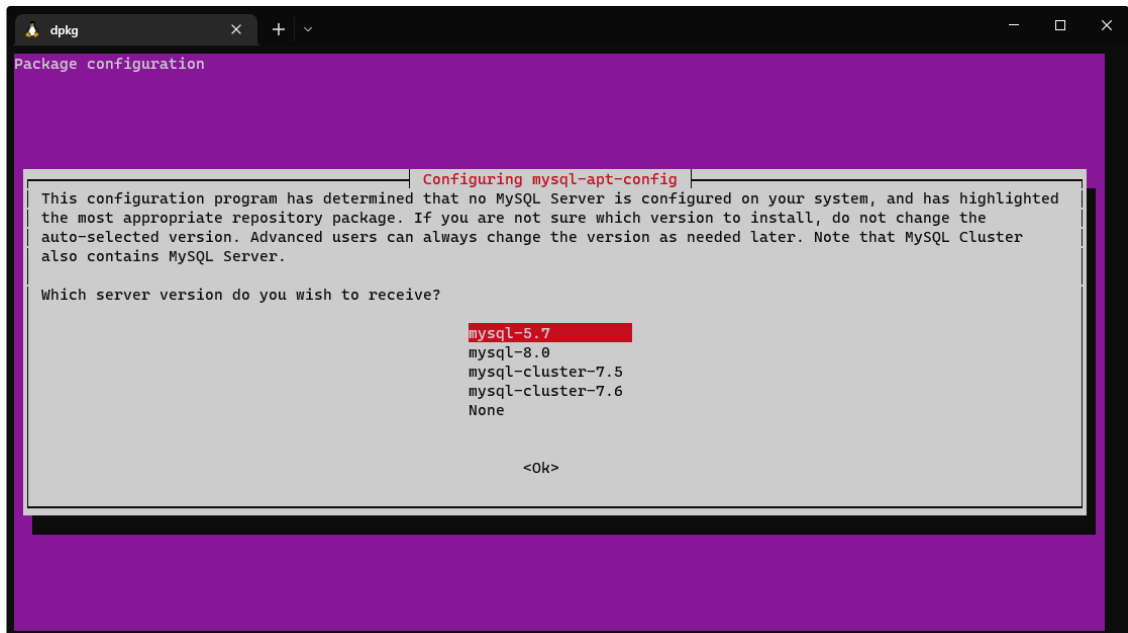
弹出框中选择: **ubuntu bionic** (Ubuntu18.04系统的代号是bionic, 选择18.04的版本库用来安装)



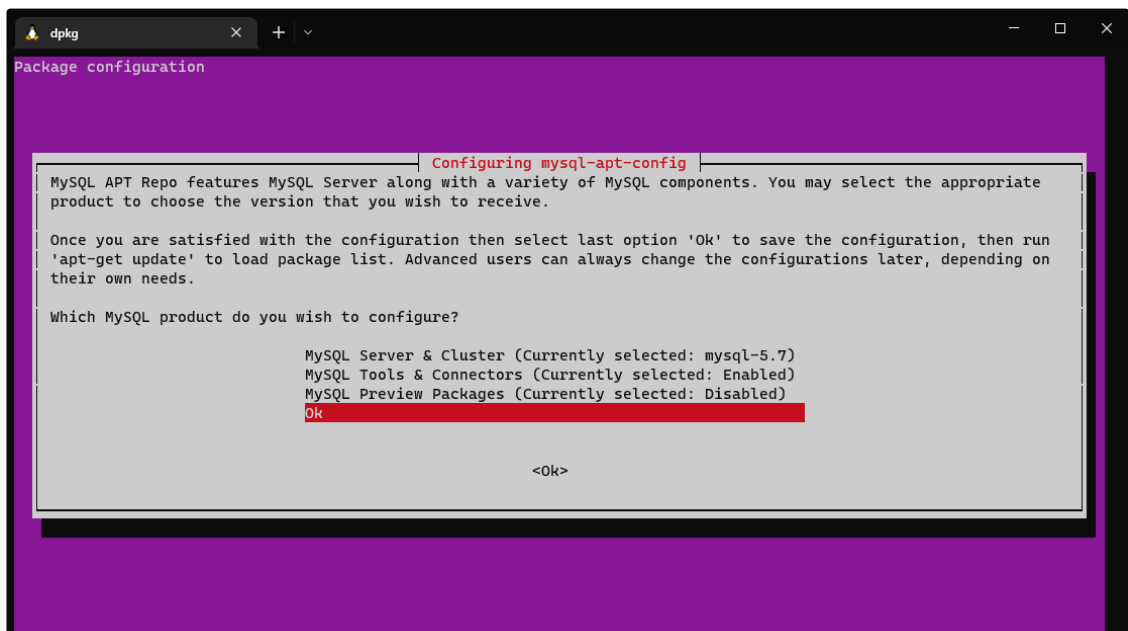
弹出框中选择: **MySQL Server & Cluster**



弹出框中选择: **mysql-5.7**



最后选择: **ok**



3. 更新apt仓库的信息

```
1 # 首先导入仓库的密钥信息
2 apt-key adv --keyserver keyserver.ubuntu.com --recv-
  keys 467B942D3A79BD29
3 # 更新仓库信息
4 apt update
```

4. 检查是否成功配置MySQL5.7的仓库

```
1 apt-cache policy mysql-server
```

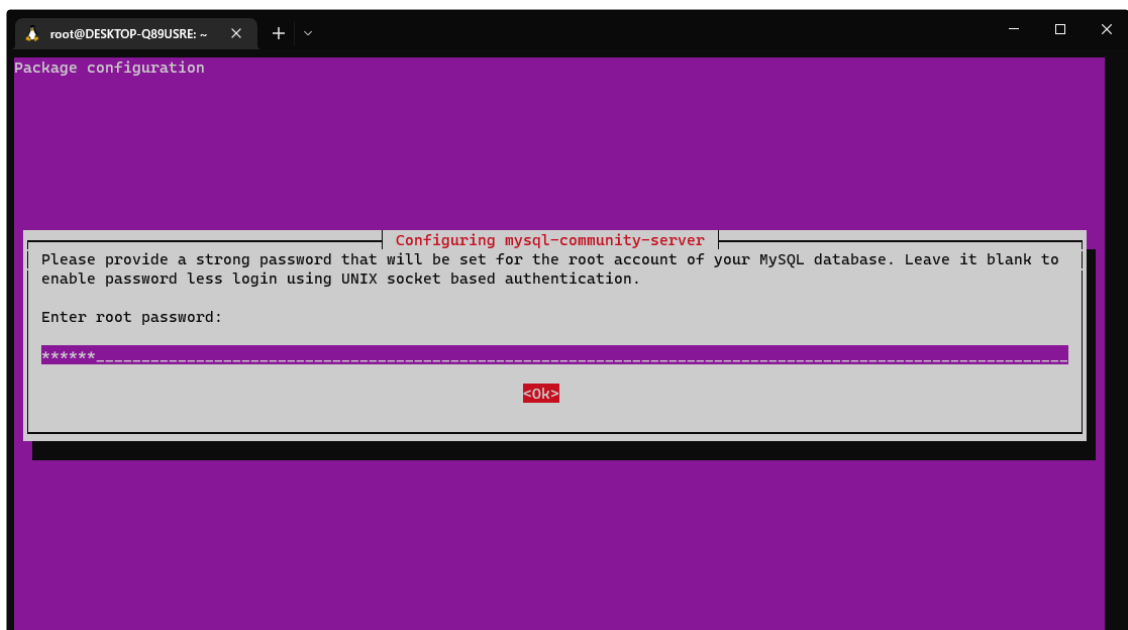
```
> sudo apt-cache policy mysql-server
mysql-server:
  Installed: (none)
  Candidate: 8.0.30-0ubuntu0.20.04.2
  Version table:
     8.0.30-0ubuntu0.20.04.2 500
        500 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages
        500 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages
     8.0.19-0ubuntu5 500
        500 http://archive.ubuntu.com/ubuntu focal/main amd64 Packages
     5.7.40-1ubuntu18.04 500
        500 http://repo.mysql.com/apt/ubuntu bionic/mysql-5.7 amd64 Packages
```

看到如图所示字样，即成功

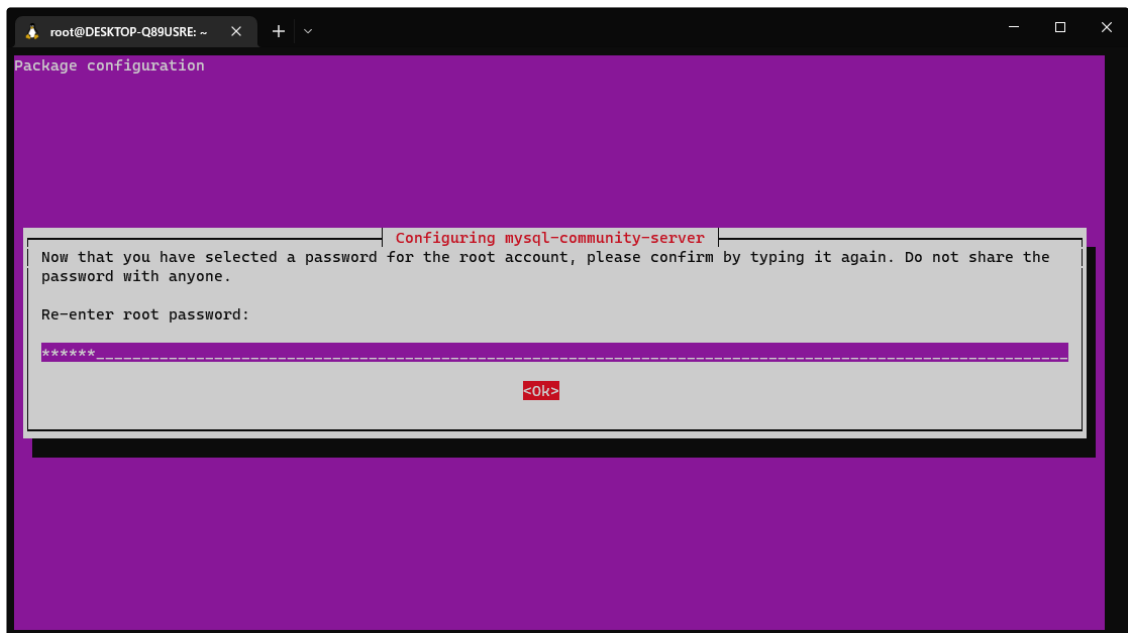
5. 安装MySQL5.7

```
1 # 使用apt安装mysql客户端和mysql服务端
2 apt install -f -y mysql-client=5.7* mysql-community-
  server=5.7*
```

弹出框中输入root密码并选择ok，密码任意，课程中以123456代替



再次输入root密码确认



6. 启动MySQL

```
1 | /etc/init.d/mysql start          # 启动
2 | /etc/init.d/mysql stop           # 停止
3 | /etc/init.d/mysql status         # 查看状态
```

```
root@DESKTOP-Q89USRE:~# /etc/init.d/mysql start
..
* MySQL Community Server 5.7.40 is started
root@DESKTOP-Q89USRE:~# /etc/init.d/mysql status
* MySQL Community Server 5.7.40 is running
```

7. 对MySQL进行初始化

```
1 | # 执行如下命令，此命令是MySQL安装后自带的配置程序
2 | mysql_secure_installation
3 | # 可以通过which命令查看到这个自带程序所在的位置
4 | root@DESKTOP-Q89USRE:~# which
   mysql_secure_installation
5 | /usr/bin/mysql_secure_installation
```

1. 输入密码:

```
root@DESKTOP-Q89USRE:~# mysql_secure_installation

Securing the MySQL server deployment.

Enter password for user root: |
```

2. 是否开启密码验证插件，如果需要增强密码安全性，输入y并回车，不需要直接回车（课程中选择直接回车）

```
VALIDATE PASSWORD PLUGIN can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD plugin?

Press y|Y for Yes, any other key for No: |
```

3. 是否更改root密码，需要输入 **y** 回车，不需要直接回车（课程不更改）

```
Using existing password for root.
Change the password for root ? ((Press y|Y for Yes, any other key for No) :
```

4. 是否移除匿名用户，移除输入 **y** 回车，不移除直接回车（课程选择移除）

```
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.
```

5. 是否进制root用户远程登录，禁止输入 **y** 回车，不禁止直接回车（课程选择不禁止）

```
Disallow root login remotely? (Press y|Y for Yes, any other key for No) :
```

6. 是否移除自带的测试数据库，移除输入 **y** 回车，不移除直接回车（课程选择不移除）

```
Remove test database and access to it? (Press y|Y for Yes, any other key for No) :
```

7. 是否刷新权限，刷新输入 **y** 回车，不刷新直接回车（课程选择刷新）

```
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
```

8. 登陆MySQL

```
1 | mysql -uroot -p
2 | # 输入密码即可登陆成功
```

```
root@DESKTOP-Q89USRE:~# mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.7.40 MySQL Community Server (GPL)

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

至此，在Ubuntu上安装MySQL5.7版本成功。

MySQL8.0版本在Ubuntu (WSL环境) 系统安装

课程中配置的WSL环境是最新的Ubuntu22.04版本，这个版本的软件商店内置的MySQL是8.0版本

所以直接可以通过apt安装即可

注意，课程是以WSL获得的Ubuntu操作系统环境。

如果你通过VMware虚拟机的方式获得了Ubuntu操作系统环境，操作步骤不用担心，和课程中使用WSL环境是 **完全一致的**

安装操作需root权限，你可以：

1. 通过 `sudo su -`，切换到root用户

课程中选择这种方式操作

2. 或在每一个命令前，加上sudo，用来临时提升权限

安装

1. 如果已经安装过MySQL5.7版本，需要卸载仓库信息哦

```
1 # 卸载MySQL5.7版本
2 apt remove -y mysql-client=5.7* mysql-community-
  server=5.7*
3
4 # 卸载5.7的仓库信息
5 dpkg -l | grep mysql | awk '{print $2}' | xargs dpkg
  -P
```

2. 更新apt仓库信息

```
1 apt update
```

3. 安装mysql

```
1 apt install -y mysql-server
```

4. 启动MySQL

```
1 /etc/init.d/mysql start          # 启动
2 /etc/init.d/mysql stop          # 停止
3 /etc/init.d/mysql status        # 查看状态
```

5. 登陆MySQL设置密码

```
1 # 直接执行: mysql
2 mysql
```

6. 设置密码

```
1 ALTER USER 'root'@'localhost' IDENTIFIED WITH
  mysql_native_password BY 'password';
```

7. 退出MySQL控制台

```
1 exit
```

8. 对MySQL进行初始化

```
1 # 执行如下命令, 此命令是MySQL安装后自带的配置程序
2 mysql_secure_installation
3 # 可以通过which命令查看到这个自带程序所在的位置
4 root@DESKTOP-Q89USRE:~# which
  mysql_secure_installation
5 /usr/bin/mysql_secure_installation
```

1. 输入密码:

```
root@DESKTOP-Q89USRE:~# mysql_secure_installation

Securing the MySQL server deployment.

Enter password for user root: |
```

2. 是否开启密码验证插件, 如果需要增强密码安全性, 输入y并回车, 不需要直接回车 (课程中选择直接回车)

```
VALIDATE PASSWORD PLUGIN can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD plugin?

Press y|Y for Yes, any other key for No: |
```

3. 是否更改root密码，需要输入 **y** 回车，不需要直接回车（课程不更改）

```
Using existing password for root.
Change the password for root ? ((Press y|Y for Yes, any other key for No) :
```

4. 是否移除匿名用户，移除输入 **y** 回车，不移除直接回车（课程选择移除）

```
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.
```

5. 是否进制root用户远程登录，禁止输入 **y** 回车，不禁止直接回车（课程选择不禁止）

```
Disallow root login remotely? (Press y|Y for Yes, any other key for No) :
```

6. 是否移除自带的测试数据库，移除输入 **y** 回车，不移除直接回车（课程选择不移除）

```
Remove test database and access to it? (Press y|Y for Yes, any other key for No) :
```

7. 是否刷新权限，刷新输入 **y** 回车，不刷新直接回车（课程选择刷新）

```
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
```

9. 重新登陆MySQL（用更改后的密码）

```
1 | mysql -uroot -p
```

```
root@DESKTOP-Q89USRE: ~  X  +  v

root@DESKTOP-Q89USRE:~# mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 17
Server version: 8.0.30-0ubuntu0.20.04.2 (Ubuntu)

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

至此，在Ubuntu上安装MySQL5.7版本成功。

Tomcat安装部署【简单】

简介

Tomcat 是由 Apache 开发的一个 Servlet 容器，实现了对 Servlet 和 JSP 的支持，并提供了作为Web服务器的一些特有功能，如Tomcat管理和控制平台、安全域管理和Tomcat阀等。

简单来说，Tomcat是一个WEB应用程序的托管平台，可以让用户编写的WEB应用程序，被Tomcat所托管，并提供网站服务。

即让用户开发的WEB应用程序，变成可以被访问的网页。

安装

Tomcat的安装非常简单，主要分为2部分：

1. 安装JDK环境
2. 解压并安装Tomcat

本次安装使用Tomcat版本是：10.0.27版本，需要Java（JDK）版本最低为JDK8或更高版本

课程中使用的JDK版本是：JDK8u351版本

安装JDK环境

1. 下载JDK软件

<https://www.oracle.com/java/technologies/downloads>

在页面下方找到：

Java SE subscribers have more choices

Also available for development, personal use, and to run other licensed Oracle products.

Java 8 Java 11

Java SE Development Kit 8u341

Java SE subscribers will receive JDK 8 updates until at least December of 2030.

The Oracle JDK 8 license changed in April 2019

The Oracle Technology Network License Agreement for Oracle Java SE is substantially different from prior Oracle JDK 8 licenses. This license permits certain uses, such as personal and development use, at no cost -- but other uses authorized under prior Oracle JDK licenses may no longer be available. Please review the terms carefully before downloading and this product. FAQs are available here.

Commercial license and support are available for a low cost with Java SE Subscription.

JDK 8 software is licensed under the Oracle Technology Network License Agreement for Oracle Java SE.

JDK 8u341 checksum

Linux macOS Solaris Windows

Product/file description	File size	Download
ARM 64 RPM Package	59.29 MB	jdk-8u341-linux-aarch64.rpm
ARM 64 Compressed Archive	71.04 MB	jdk-8u341-linux-aarch64.tar.gz
ARM 32 Hard Float ABI	73.75 MB	jdk-8u341-linux-arm32-vfp-hflt.tar.gz
x86 RPM Package	111.11 MB	jdk-8u341-linux-i586.rpm
x86 Compressed Archive	140.83 MB	jdk-8u341-linux-i586.tar.gz
x64 RPM Package	110.95 MB	jdk-8u341-linux-x64.rpm
x64 Compressed Archive	141.30 MB	jdk-8u341-linux-x64.tar.gz

下载 `jdk-8u351-linux-x64.tar.gz`

You must accept the [Oracle Technology Network License Agreement for Oracle Java SE](#) to download this software.

☒ I reviewed and accept the Oracle Technology Network License Agreement for Oracle Java SE

Required

Download `jdk-8u341-linux-x64.tar.gz`

在弹出的页面中输入Oracle的账户密码即可下载（如无账户，请自行注册，注册是免费的）

2. 登陆Linux系统，切换到root用户

```
[itheima@centos ~]$ su -
密码:
上一次登录: 日 10月 16 16:02:24 CST 2022pts/0 上
[root@centos ~]#
```

3. 通过FinalShell，上传下载好的JDK安装包

```
[itheima@centos ~]$ su -
密码:
上一次登录: 日 10月 16 16:02:24 CST 2022pts/0 上
[root@centos ~]# rz
```

路由追踪 | 进程管理 | 网络监控 | 速度测试
自定义加速 | 双边TCP加速
下载: C:\Users\javac\Desktop\fsdownload
jdk-8u341-linux-x64.tar.gz 25.1% 4.3M/s 141.3M

4. 创建文件夹，用来部署JDK，将JDK和Tomcat都安装部署到: /export/server 内

```
1 | mkdir -p /export/server
```

5. 解压缩JDK安装文件

```
1 | tar -zxvf jdk-8u351-linux-x64.tar.gz -C  
    /export/server
```

6. 配置JDK的软链接

```
1 | ln -s /export/server/jdk1.8.0_351 /export/server/jdk
```

7. 配置JAVA_HOME环境变量, 以及将\$JAVA_HOME/bin文件夹加入PATH环境变量中

```
1 | # 编辑/etc/profile文件  
2 | export JAVA_HOME=/export/server/jdk  
3 | export PATH=$PATH:$JAVA_HOME/bin
```

8. 生效环境变量

```
1 | source /etc/profile
```

9. 配置java执行程序的软链接

```
1 | # 删除系统自带的java程序  
2 | rm -f /usr/bin/java  
3 | # 软链接我们自己安装的java程序  
4 | ln -s /export/server/jdk/bin/java /usr/bin/java
```

10. 执行验证:

```
1 | java -version  
2 | javac -version
```


解压并部署Tomcat

Tomcat建议使用非Root用户安装并启动

可以创建一个用户：tomcat用以部署

1. 首先，放行tomcat需要使用的8080端口的的外部访问权限

CentOS系统默认开启了防火墙，阻止外部网络流量访问系统内部

所以，如果想要Tomcat可以正常使用，需要对Tomcat默认使用的8080端口进行放行

放行有2种操作方式：

1. 关闭防火墙
2. 配置防火墙规则，放行端口

```
1 # 以下操作2选一即可
2 # 方式1：关闭防火墙
3 systemctl stop firewalld          # 关闭防火墙
4 systemctl disable firewalld       # 停止防火墙开机自启
5
6 # 方式2：放行8080端口的外部访问
7 firewall-cmd --add-port=8080/tcp --permanent
  # --add-port=8080/tcp表示放行8080端口的tcp访问，--
  permanent表示永久生效
8 firewall-cmd --reload
  # 重新载入防火墙规则使其生效
```

方便起见，建议同学们选择方式1，直接关闭防火墙一劳永逸

防火墙的配置非常复杂，后面会视情况独立出一集防火墙配置规则的章节。

2. 以root用户操作，创建tomcat用户

```
1 # 使用root用户操作
2 useradd tomcat
3 # 可选，为tomcat用户配置密码
4 passwd tomcat
```

3. 下载Tomcat安装包

```
1 # 使用root用户操作
2 wget https://dlcdn.apache.org/tomcat/tomcat-
  10/v10.0.27/bin/apache-tomcat-10.0.27.tar.gz
3 # 如果出现https相关错误, 可以使用--no-check-certificate选项
4 wget --no-check-certificate
  https://dlcdn.apache.org/tomcat/tomcat-
  10/v10.0.27/bin/apache-tomcat-10.0.27.tar.gz
```

如果Linux内下载过慢, 可以复制下载链接在Windows系统中使用迅雷等软件加速下载然后上传到Linux内即可
或者使用课程资料中提供的安装包

4. 解压Tomcat安装包

```
1 # 使用root用户操作, 否则无权限解压到/export/server内, 除非
  修改此文件夹权限
2 tar -zxvf apache-tomcat-10.0.27.tar.gz -C
  /export/server
```

5. 创建Tomcat软链接

```
1 # 使用root用户操作
2 ln -s /export/server/apache-tomcat-10.0.27
  /export/server/tomcat
```

6. 修改tomcat安装目录权限

```
1 # 使用root用户操作, 同时对软链接和tomcat安装文件夹进行修改,
  使用通配符*进行匹配
2 chown -R tomcat:tomcat /export/server/*tomcat*
```

7. 切换到tomcat用户

```
1 su - tomcat
```

8. 启动tomcat

```
1 | /export/server/tomcat/bin/startup.sh
```

9. tomcat启动在8080端口, 可以检查是否正常启动成功

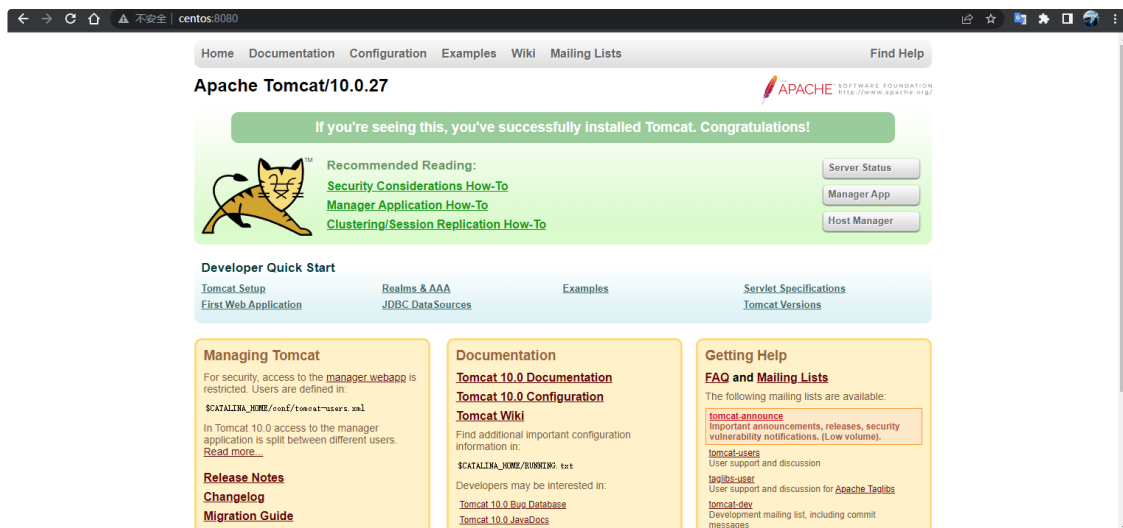
```
1 | netstat -anp | grep 8080
```

```
[tomcat@centos ~]$ netstat -anp | grep 8080
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
tcp6      0      0 :::8080          :::*              LISTEN      68811/java
unix      3      [ ]              STREAM          CONNECTED       48080      -           /run/systemd/journal/stdout
```

10. 打开浏览器, 输入:

<http://centos:8080>或<http://192.168.88.130:8080>

使用主机名 (需配置好本地的主机名映射) 或IP地址访问Tomcat的WEB页面



至此, Tomcat安装配置完成。

Nginx安装部署【简单】

简介

Nginx (engine x) 是一个高性能的HTTP和反向代理web服务器, 同时也提供了IMAP/POP3/SMTP服务。

同Tomcat一样，Nginx可以托管用户编写的WEB应用程序成为可访问的网页服务，同时也可以作为流量代理服务器，控制流量的中转。

Nginx在WEB开发领域，基本上也是必备组件之一了。

安装

Nginx同样需要配置额外的yum仓库，才可以使用yum安装

安装Nginx的操作需要root身份

1. 安装yum依赖程序

```
1 # root执行
2 yum install -y yum-utils
```

2. 手动添加，nginx的yum仓库

yum程序使用的仓库配置文件，存放在：`/etc/yum.repo.d`内。

```
1 # root执行
2 # 创建文件使用vim编辑
3 vim /etc/yum.repos.d/nginx.repo
4 # 填入如下内容并保存退出
5 [nginx-stable]
6 name=nginx stable repo
7 baseurl=http://nginx.org/packages/centos/$releasever/$basearch/
8 gpgcheck=1
9 enabled=1
10 gpgkey=https://nginx.org/keys/nginx_signing.key
11 module_hotfixes=true
12
13 [nginx-mainline]
14 name=nginx mainline repo
15 baseurl=http://nginx.org/packages/mainline/centos/$releasever/$basearch/
```

```
16 | gpgcheck=1
17 | enabled=0
18 | gpgkey=https://nginx.org/keys/nginx_signing.key
19 | module_hotfixes=true
```

通过如上操作，我们手动添加了nginx的yum仓库

3. 通过yum安装最新稳定版的nginx

```
1 | # root执行
2 | yum install -y nginx
```

4. 启动

```
1 | # nginx自动注册了systemctl系统服务
2 | systemctl start nginx          # 启动
3 | systemctl stop nginx           # 停止
4 | systemctl status nginx         # 运行状态
5 | systemctl enable nginx         # 开机自启
6 | systemctl disable nginx        # 关闭开机自启
```

5. 配置防火墙放行

nginx默认绑定80端口，需要关闭防火墙或放行80端口

```
1 | # 方式1（推荐），关闭防火墙
2 | systemctl stop firewalld        # 关闭
3 | systemctl disable firewalld     # 关闭开机自启
4 |
5 | # 方式2，放行80端口
6 | firewall-cmd --add-port=80/tcp --permanent # 放行tcp规则下的80端口，永久生效
7 | firewall-cmd --reload           # 重新加载防火墙规则
```

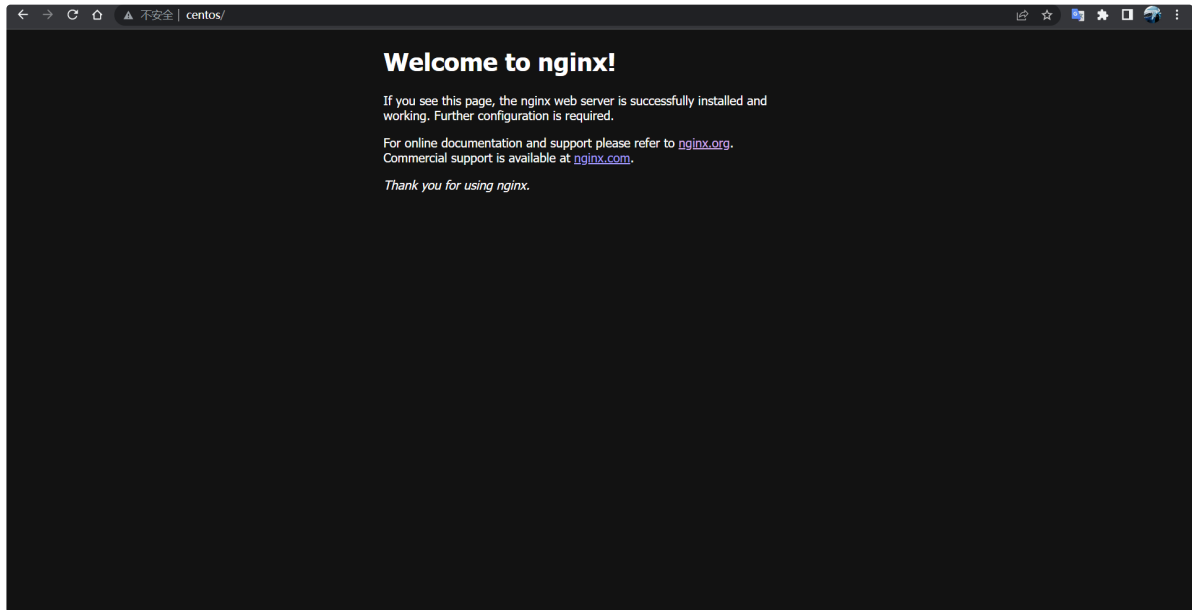
6. 启动后浏览器输入Linux服务器的IP地址或主机名即可访问

<http://192.168.88.130> 或 <http://centos>

ps: 80端口是访问网站的默认端口，所以后面无需跟随端口号显示的指定端口也是可以的比如：

- <http://192.168.88.130:80>
- <http://centos:80>

至此，Nginx安装配置完成。



RabbitMQ安装部署【简单】

简介

RabbitMQ一款知名的开源消息队列系统，为企业提供消息的发布、订阅、点对点传输等消息服务。

RabbitMQ在企业开发中十分常见，课程为大家演示快速搭建RabbitMQ环境。

安装

rabbitmq在yum仓库中的版本比较老，所以我们需要手动构建yum仓库

1. 准备yum仓库

```
1 # root执行
2 # 1. 准备gpgkey密钥
3 rpm --import https://github.com/rabbitmq/signing-
  keys/releases/download/2.0/rabbitmq-release-
  signing-key.asc
4 rpm --import
  https://packagecloud.io/rabbitmq/erlang/gpgkey
5 rpm --import
  https://packagecloud.io/rabbitmq/rabbitmq-
  server/gpgkey
6
7 # 2. 准备仓库文件
8 vim /etc/yum.repos.d/rabbitmq.repo
9 # 填入如下内容
10 ##
11 ## Zero dependency Erlang
12 ##
13
14 [rabbitmq_erlang]
15 name=rabbitmq_erlang
16 baseurl=https://packagecloud.io/rabbitmq/erlang/el/
  7/$basearch
17 repo_gpgcheck=1
18 gpgcheck=1
19 enabled=1
20 # PackageCloud's repository key and RabbitMQ
  package signing key
21 gpgkey=https://packagecloud.io/rabbitmq/erlang/gpgk
  ey
22      https://github.com/rabbitmq/signing-
  keys/releases/download/2.0/rabbitmq-release-
  signing-key.asc
23 sslverify=1
24 sslcacert=/etc/pki/tls/certs/ca-bundle.crt
25 metadata_expire=300
26
27 [rabbitmq_erlang-source]
28 name=rabbitmq_erlang-source
```

```
29 baseurl=https://packagecloud.io/rabbitmq/erlang/el/
   7/SRPMS
30 repo_gpgcheck=1
31 gpgcheck=0
32 enabled=1
33 # PackageCloud's repository key and RabbitMQ
   package signing key
34 gpgkey=https://packagecloud.io/rabbitmq/erlang/gpgk
   ey
35     https://github.com/rabbitmq/signing-
   keys/releases/download/2.0/rabbitmq-release-
   signing-key.asc
36 sslverify=1
37 sslcacert=/etc/pki/tls/certs/ca-bundle.crt
38 metadata_expire=300
39
40 ##
41 ## RabbitMQ server
42 ##
43
44 [rabbitmq_server]
45 name=rabbitmq_server
46 baseurl=https://packagecloud.io/rabbitmq/rabbitmq-
   server/el/7/$basearch
47 repo_gpgcheck=1
48 gpgcheck=0
49 enabled=1
50 # PackageCloud's repository key and RabbitMQ
   package signing key
51 gpgkey=https://packagecloud.io/rabbitmq/rabbitmq-
   server/gpgkey
52     https://github.com/rabbitmq/signing-
   keys/releases/download/2.0/rabbitmq-release-
   signing-key.asc
53 sslverify=1
54 sslcacert=/etc/pki/tls/certs/ca-bundle.crt
55 metadata_expire=300
56
57 [rabbitmq_server-source]
58 name=rabbitmq_server-source
```



```
59 baseUrl=https://packagecloud.io/rabbitmq/rabbitmq-  
server/el/7/SRPMS  
60 repo_gpgcheck=1  
61 gpgcheck=0  
62 enabled=1  
63 gpgkey=https://packagecloud.io/rabbitmq/rabbitmq-  
server/gpgkey  
64 sslverify=1  
65 sslcacert=/etc/pki/tls/certs/ca-bundle.crt  
66 metadata_expire=300
```

2. 安装RabbitMQ

```
1 # root执行  
2 yum install erlang rabbitmq-server -y
```

```
1 Installed:  
2 erlang.x86_64 0:23.3.4.11-1.el7  
  rabbitmq-server.noarch 0:3.10.0-1.el7
```

3. 启动

```
1 # root执行  
2 # 使用systemctl管控, 服务名: rabbitmq-server  
3 systemctl enable rabbitmq-server          # 开机自启  
4 systemctl disable rabbitmq-server         # 关闭开机自启  
5 systemctl start rabbitmq-server           # 启动  
6 systemctl stop rabbitmq-server            # 关闭  
7 systemctl status rabbitmq-server          # 查看状态
```

4. 放行防火墙, RabbitMQ使用5672、15672、25672 3个端口

```

1 # 方式1 (推荐) , 关闭防火墙
2 systemctl stop firewalld          # 关闭
3 systemctl disable firewalld        # 关闭开机自启
4
5 # 方式2, 放行5672 25672端口
6 firewall-cmd --add-port=5672/tcp --permanent
  # 放行tcp规则下的5672端口, 永久生效
7 firewall-cmd --add-port=15672/tcp --permanent
  # 放行tcp规则下的15672端口, 永久生效
8 firewall-cmd --add-port=25672/tcp --permanent
  # 放行tcp规则下的25672端口, 永久生效
9 firewall-cmd --reload
  # 重新加载防火墙规则

```

5. 启动RabbitMQ的WEB管理控制台

```

1 rabbitmq-plugins enable rabbitmq_management

```

6. 添加admin用户, 并赋予权限

```

1 rabbitmqctl add_user admin 'Itheima66^'
2 rabbitmqctl set_permissions -p "/" "admin" ".*" ".*"
  ".*"
3 rabbitmqctl set_user_tags admin administrator

```

7. 浏览器打开管理控制台

<http://192.168.88.130:15672>

The screenshot shows the RabbitMQ Management UI. At the top, it says 'RabbitMQ 3.10.0 Erlang 23.3.4.11'. The 'Overview' tab is selected, showing 'Totals' for Queued messages, Currently idle, Message rates, and Global counts. Below this, there's a 'Nodes' section with a table of system statistics for the 'rabbit@centos' node. The table includes metrics like File descriptors, Socket descriptors, Erlang processes, Memory, Disk space, and Uptime. At the bottom, there are links for HTTP API, Server Docs, Tutorials, and other resources.

Name	File descriptors	Socket descriptors	Erlang processes	Memory	Disk space	Uptime	Info	Reset stats
rabbit@centos	36	0	351	67 MiB	12 GiB	8m 13s	basic disc 1 rrs	This node All nodes

至此，RabbitMQ已经安装完成了。

Redis安装部署【简单】

简介

redis是一个开源的、使用C语言编写的、支持网络交互的、可基于内存也可持久化的Key-Value数据库。

redis的特点就是：**快**，可以基于内存存储数据并提供超低延迟、超快的检索速度
一般用于在系统中提供快速缓存的能力。

安装

1. 配置EPEL仓库

EPEL 的全称叫 Extra Packages for Enterprise Linux 。EPEL 是由 Fedora 社区打造，为 RHEL 及衍生发行版如 CentOS、Scientific Linux 等提供高质量软件包的项目。装上了 EPEL之后，就相当于添加了一个第三方源。EPEL则为服务器版本提供大量的rpm包（yum程序所使用的程序安装包，类似Windows的exe），而且大多数rpm包在官方 repository 中是找不到的。

```
1 # root执行
2 yum install -y epel-release
```

2. 安装redis

```
1 # root执行
2 yum install -y redis
```

3. 启动redis

```
1 # root执行
2 # 使用systemctl管控, 服务名: redis
3 systemctl enable redis      # 开机自启
4 systemctl disable redis     # 关闭开机自启
5 systemctl start redis       # 启动
6 systemctl stop redis        # 关闭
7 systemctl status redis      # 查看状态
```

4. 放行防火墙, redis使用端口6379

```
1 # 方式1 (推荐), 关闭防火墙
2 systemctl stop firewalld      # 关闭
3 systemctl disable firewalld   # 关闭开机自启
4
5 # 方式2, 放行6379端口
6 firewall-cmd --add-port=6379/tcp --permanent
   # 放行tcp规则下的6379端口, 永久生效
7 firewall-cmd --reload
```

5. 进入redis服务

```
1 # 执行redis-cli
2 [root@centos ~]# redis-cli
3 127.0.0.1:6379> set mykey hello
4 OK
5 127.0.0.1:6379> get mykey
6 "hello"
7 127.0.0.1:6379>
```

至此, redis安装完成。

ElasticSearch安装部署

简介

全文搜索属于最常见的需求，开源的 **Elasticsearch**（以下简称 es）是目前全文搜索引擎的首选。

它可以快速地储存、搜索和分析海量数据。维基百科、Stack Overflow、Github 都采用它。

Elasticsearch简称es，在企业内同样是一款应用非常广泛的搜索引擎服务。

很多服务中的搜索功能，都是基于es来实现的。

安装

1. 添加yum仓库

```
1 # root执行
2 # 导入仓库密钥
3 rpm --import https://artifacts.elastic.co/GPG-KEY-
  elasticsearch
4
5 # 添加yum源
6 # 编辑文件
7 vim /etc/yum.repos.d/elasticsearch.repo
8
9 [elasticsearch-7.x]
10 name=Elasticsearch repository for 7.x packages
11 baseurl=https://artifacts.elastic.co/packages/7.x/y
  um
12 gpgcheck=1
13 gpgkey=https://artifacts.elastic.co/GPG-KEY-
  elasticsearch
14 enabled=1
15 autorefresh=1
16 type=rpm-md
17
18
19 # 更新yum缓存
```

```
20 yum makecache
```

2. 安装es

```
1 yum install -y elasticsearch
```

3. 配置es

```
1 vim /etc/elasticsearch/elasticsearch.yml
2
3 # 17行, 设置集群名称
4 cluster.name: my-cluster
5
6 # 23行, 设置节点名称
7 node.name: node-1
8
9 # 56行, 允许外网访问
10 network.host: 0.0.0.0
11
12 # 74行, 配置集群master节点
13 cluster.initial_master_nodes: ["node-1"]
```

4. 启动es

```
1 systemctl start | stop | status | enable | disable
   elasticsearch
```

5. 关闭防火墙

```
1 systemctl stop firewalld
2 systemctl disable firewalld
```

6. 测试

浏览器打开: <http://ip:9200/?pretty>

```
{
  "name" : "node-1",
  "cluster_name" : "my-application",
  "cluster_uuid" : "DRbuWXEBQlOVrmLYiRpGAg",
  "version" : {
    "number" : "7.17.6",
    "build_flavor" : "default",
    "build_type" : "rpm",
    "build_hash" : "f65e9d338dc1d07b642e14a27f338990148ee5b6",
    "build_date" : "2022-08-23T11:08:48.893373482Z",
    "build_snapshot" : false,
    "lucene_version" : "8.11.1",
    "minimum_wire_compatibility_version" : "6.8.0",
    "minimum_index_compatibility_version" : "6.0.0-beta1"
  },
  "tagline" : "You Know, for Search"
}
```

集群化环境前置准备

介绍

在前面，我们所学习安装的软件，都是以单机模式运行的。

后续，我们将要学习大数据相关的软件部署，所以后续我们所安装的软件服务，大多数都是以集群化（多台服务器共同工作）模式运行的。

所以，在当前小节，我们需要完成集群化环境的前置准备，包括创建多台虚拟机，配置主机名映射，SSH免密登录等等。

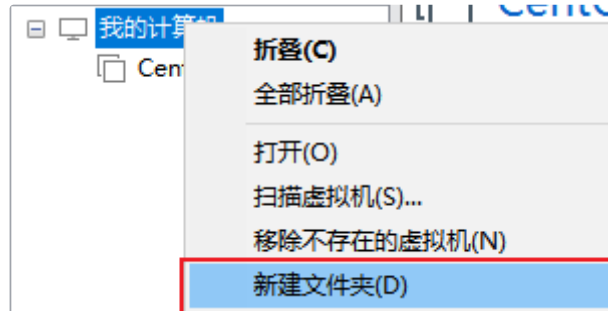
部署

配置多台Linux虚拟机

安装集群化软件，首要条件就是要有多台Linux服务器可用。

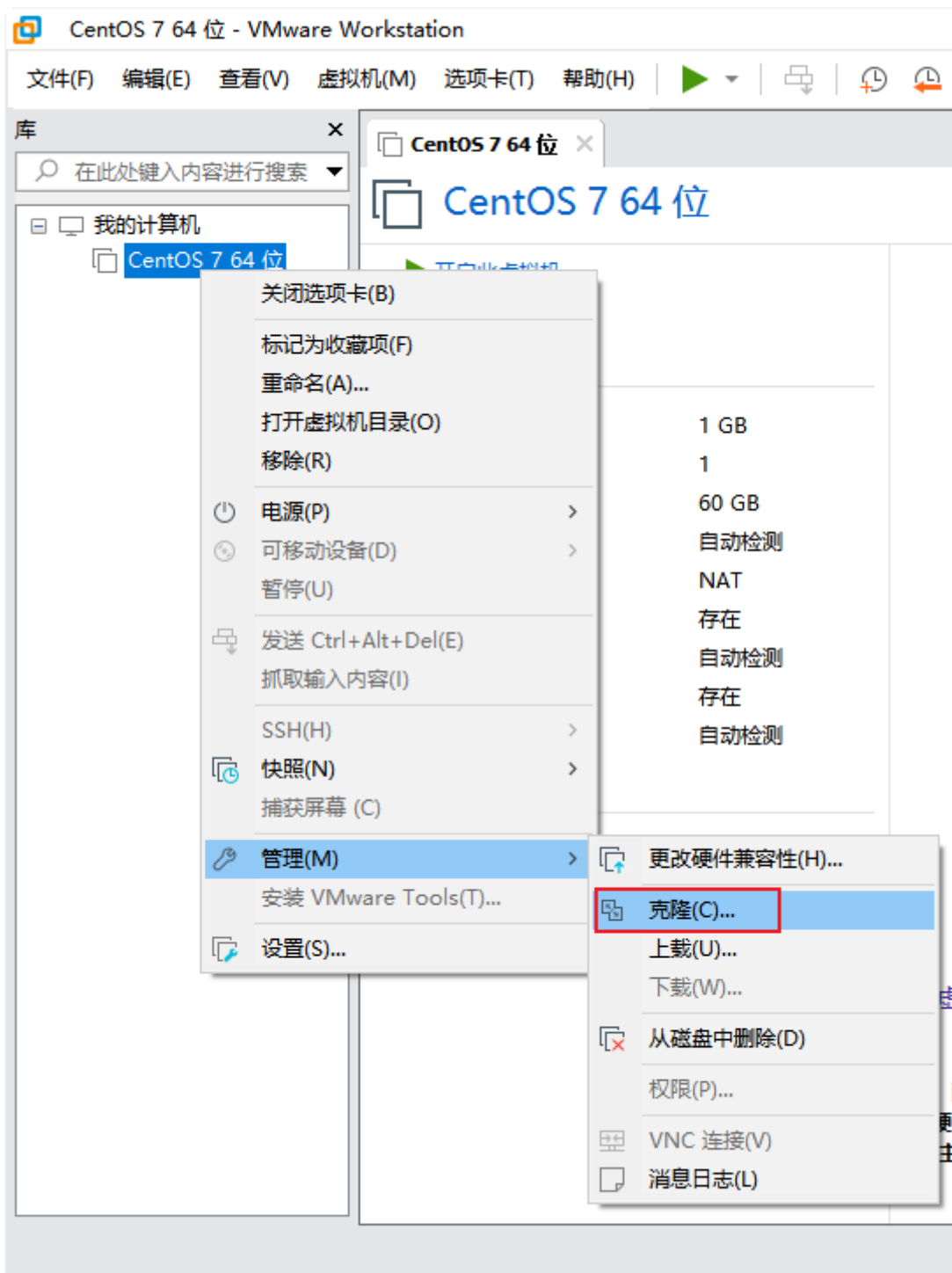
我们可以使用VMware提供的克隆功能，将我们的虚拟机额外克隆出3台来使用。

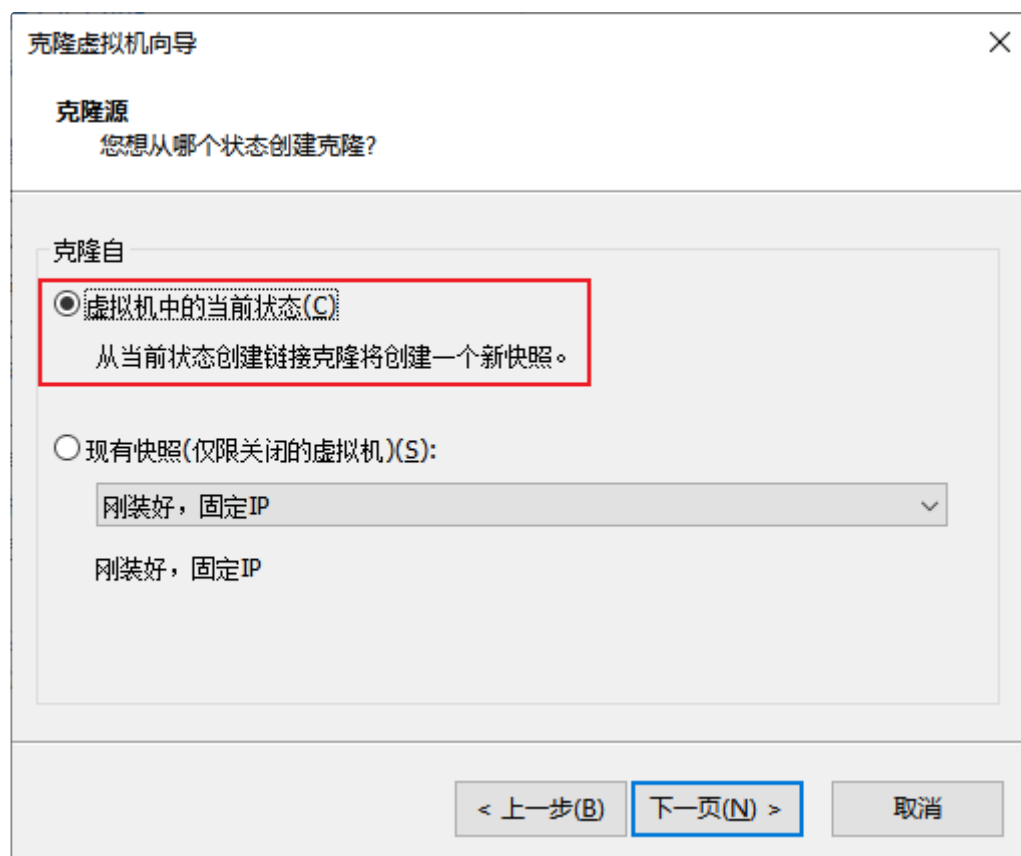
1. 首先，关机当前CentOS系统虚拟机（可以使用root用户执行 `init 0` 来快速关机）
2. 新建文件夹



文件夹起名为：虚拟机集群

3. 克隆





克隆虚拟机向导

克隆类型
您希望如何克隆此虚拟机?

克隆方法

☐ 创建链接克隆(L)
链接克隆是对原始虚拟机的引用，所需的存储磁盘空间较少。但是，必须能够访问原始虚拟机才能运行。

☒ 创建完整克隆(F)
完整克隆是原始虚拟机当前状态的完整副本。此副本虚拟机完全独立，但需要较多的存储磁盘空间。

< 上一步(B)

下一页(N) >

取消

克隆虚拟机向导

新虚拟机名称
您希望该虚拟机使用什么名称?

虚拟机名称(V)
node1

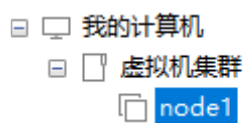
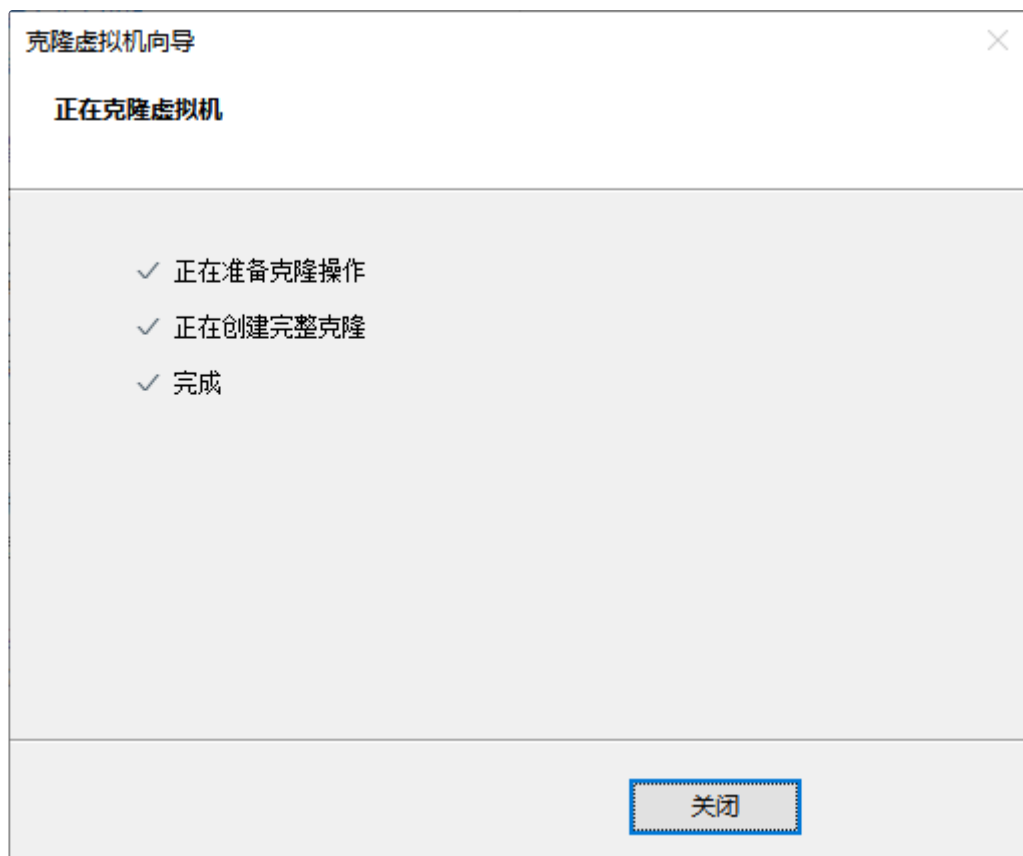
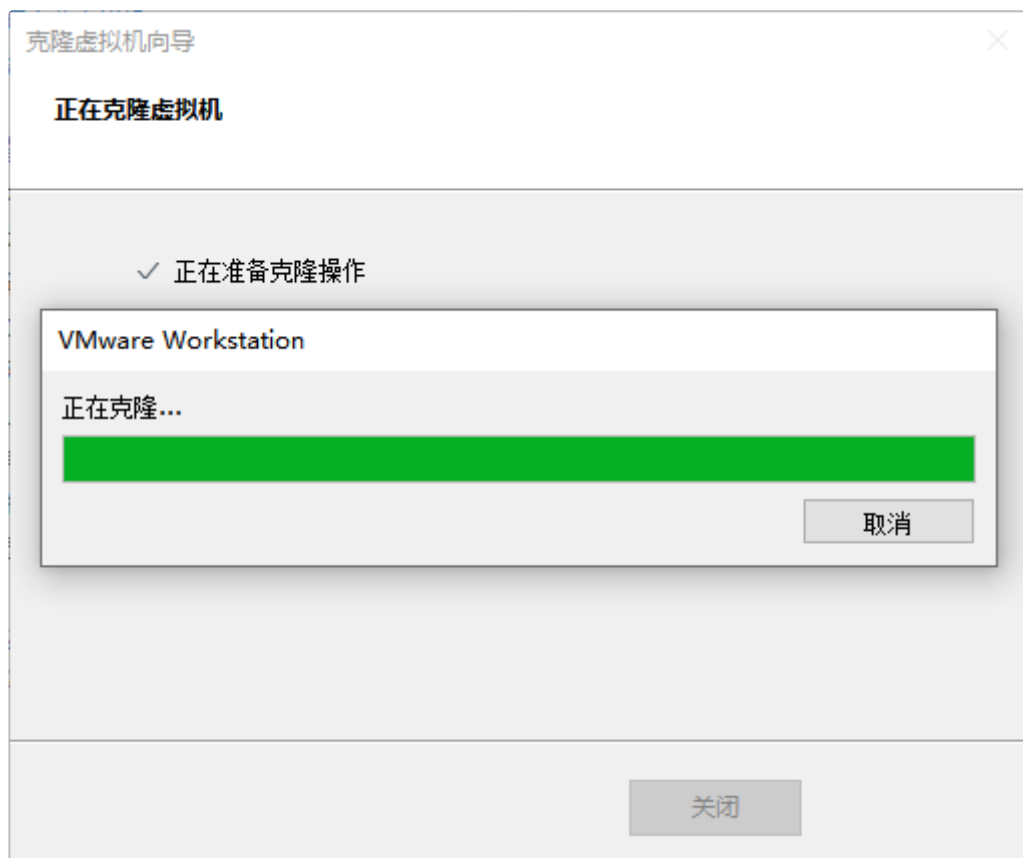
位置(L)
E:\vm\cluster\node1

浏览(R)...

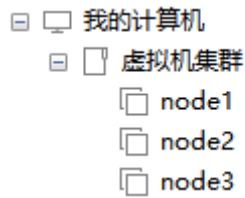
< 上一步(B)

完成

取消



4. 同样的操作克隆出：node2和node3



5. 开启node1, 修改主机名为node1, 并修改固定ip为: 192.168.88.131

```
1 # 修改主机名
2 hostnamectl set-hostname node1
3
4 # 修改IP地址
5 vim /etc/sysconfig/network-scripts/ifcfg-ens33
6 IPADDR="192.168.88.131"
7
8 # 重启网卡
9 systemctl stop network
10 systemctl start network
11 # 或者直接
12 systemctl restart network
```

6. 同样的操作启动node2和node3,

修改node2主机名为node2, 设置ip为192.168.88.132

修改node2主机名为node3, 设置ip为192.168.88.133

7. 配置FinalShell, 配置连接到node1、node2、node3的连接

为了简单起见, 建议配置root用户登录

准备主机名映射

1. 在Windows系统中修改hosts文件, 填入如下内容:

如果同学们使用MacOS系统, 请:

1. `sudo su -`, 切换到root
2. 修改/etc/hosts文件

```
1 192.168.88.131 node1
2 192.168.88.132 node2
3 192.168.88.133 node3
```

2. 在3台Linux的/etc/hosts文件中，填入如下内容（**3台都要添加**）

```
1 192.168.88.131 node1
2 192.168.88.132 node2
3 192.168.88.133 node3
```

配置SSH免密登录

简介

SSH服务是一种用于远程登录的安全认证协议。

我们通过FinalShell远程连接到Linux，就是使用的SSH服务。

SSH服务支持：

1. 通过账户+密码的认证方式来做用户认证
2. 通过账户+秘钥文件的方式做用户认证

SSH可以让我们通过SSH命令，远程的登陆到其它的主机上，比如：

在node1执行：ssh root@node2，将以root用户登录node2服务器，输入密码即可成功登陆

或者ssh node2，将以当前用户直接登陆到node2服务器。

SSH免密配置

后续安装的集群化软件，多数需要远程登录以及远程执行命令，我们可以简单起见，配置三台Linux服务器之间的免密码互相SSH登陆

1. 在每一台机器都执行：ssh-keygen -t rsa -b 4096，一路回车到底即可
2. 在每一台机器都执行：

```
1 | ssh-copy-id node1
2 | ssh-copy-id node2
3 | ssh-copy-id node3
```

3. 执行完毕后，node1、node2、node3之间将完成root用户之间的免密互通

配置JDK环境

后续的大数据集群软件，多数是需要Java运行环境的，所以我们为 **每一台** 机器都配置JDK环境。

JDK配置参阅：[Tomcat](#) 安装部署环节。

关闭防火墙和SELinux

集群化软件之间需要通过端口互相通讯，为了避免出现网络不通的问题，我们可以简单的在集群内部关闭防火墙。

在每一台机器都执行

```
1 | systemctl stop firewalld
2 | systemctl disable firewalld
```

Linux有一个安全模块：SELinux，用以限制用户和程序的相关权限，来确保系统的安全稳定。

SELinux的配置同防火墙一样，非常复杂，课程中不多涉及，后续视情况可以出一章SELinux的配置课程。

在当前，我们只需要关闭SELinux功能，避免导致后面的软件运行出现问题即可，

在每一台机器都执行

```
1 vim /etc/sysconfig/selinux
2
3 # 将第七行, SELINUX=enforcing 改为
4 SELINUX=disabled
5 # 保存退出后, 重启虚拟机即可, 千万要注意disabled单词不要写错, 不然无法启动系统
```

添加快照

为了避免后续出现问题, 在完成上述设置后, 为 **每一台虚拟机** 都制作快照, 留待使用。

补充命令 - scp

后续的安装部署操作, 我们将会频繁的在多台服务器之间相互传输数据。

为了更加方便的互相传输, 我们补充一个命令: scp

scp命令是cp命令的升级版, 即: ssh cp, 通过SSH协议完成文件的复制。

其主要的功能就是: 在不同的Linux服务器之间, 通过 **SSH** 协议互相传输文件。

只要知晓服务器的账户和密码 (或密钥), 即可通过SCP互传文件。

语法:

```
1 scp [-r] 参数1 参数2
2 - -r选项用于复制文件夹使用, 如果复制文件夹, 必须使用-r
3 - 参数1: 本机路径 或 远程目标路径
4 - 参数2: 远程目标路径 或 本机路径
5
6 如:
7 scp -r /export/server/jdk root@node2:/export/server/
```



```
8  将本机上的jdk文件夹， 以root的身份复制到node2
   的/export/server/内
9  同SSH登陆一样，账户名可以省略（使用本机当前的同名账户登陆）
10
11 如：
12 scp -r node2:/export/server/jdk /export/server/
13 将远程node2的jdk文件夹，复制到本机的/export/server/内
14
15
16 # scp命令的高级用法
17 cd /export/server
18 scp -r jdk node2:`pwd` /      # 将本机当前路径的jdk文件夹，复
   制到node2服务器的同名路径下
19 scp -r jdk node2:$PWD        # 将本机当前路径的jdk文件夹，复
   制到node2服务器的同名路径下
```

Zookeeper集群安装部署

简介

ZooKeeper是一个分布式的，开放源码的分布式应用程序协调服务，是Hadoop和Hbase的重要组件。它是一个为分布式应用提供一致性服务的软件，提供的功能包括：配置维护、域名服务、分布式同步、组服务等。

除了为Hadoop和HBase提供协调服务外，Zookeeper也被其它许多软件采用作为其分布式状态一致性的依赖，比如Kafka，又或者一些软件项目中，也经常能见到Zookeeper作为一致性协调服务存在。

Zookeeper不论是大数据领域亦或是其它服务器开发领域，涉及到分布式状态一致性的场景，总有它的身影存在。

安装

Zookeeper是一款分布式的集群化软件，可以在多台服务器上部署，并协同组成分布式集群一起工作。

1. 首先，要确保已经完成了**集群化环境前置准备**环节的全部内容
2. 【node1上操作】下载Zookeeper安装包，并解压

```
1 # 下载
2 wget
  http://archive.apache.org/dist/zookeeper/zookeeper-
  3.5.9/apache-zookeeper-3.5.9-bin.tar.gz
3
4 # 确保如下目录存在，不存在就创建
5 mkdir -p /export/server
6
7 # 解压
8 tar -zxvf apache-zookeeper-3.5.9-bin.tar.gz -C
  /export/server
```

3. 【node1上操作】创建软链接

```
1 ln -s /export/server/apache-zookeeper-3.5.9
  /export/server/zookeeper
```

4. 【node1上操作】修改配置文件

```
1 vim /export/server/zookeeper/conf/zoo.cfg
2
3 tickTime=2000
4 # zookeeper数据存储目录
5 dataDir=/export/server/zookeeper/data
6 clientPort=2181
7 initLimit=5
8 syncLimit=2
9 server.1=node1:2888:3888
10 server.2=node2:2888:3888
11 server.3=node3:2888:3888
```

5. 【node1上操作】配置myid

```
1 # 1. 创建Zookeeper的数据目录
2 mkdir /export/server/zookeeper/data
3
4 # 2. 创建文件，并填入1
5 vim /export/server/zookeeper/data/myid
6 # 在文件内填入1即可
```

6. 【在node2和node3上操作】，创建文件夹

```
1 mkdir -p /export/server
```

7. 【node1上操作】将Zookeeper 复制到node2和node3

```
1 cd /export/server
2
3 scp -r apache-zookeeper-3.5.9 node2:`pwd`/
4 scp -r apache-zookeeper-3.5.9 node3:`pwd`/
```

8. 【在node2上操作】

```
1 # 1. 创建软链接
2 ln -s /export/server/apache-zookeeper-3.5.9
  /export/server/zookeeper
3
4 # 2. 修改myid文件
5 vim /export/server/zookeeper/data/myid
6 # 修改内容为2
```

9. 【在node3上操作】

```
1 # 1. 创建软链接
2 ln -s /export/server/apache-zookeeper-3.5.9
  /export/server/zookeeper
3
4 # 2. 修改myid文件
5 vim /export/server/zookeeper/data/myid
6 # 修改内容为3
```

10. 【在node1、node2、node3上分别执行】 启动Zookeeper

```
1 # 启动命令
2 /export/server/zookeeper/bin/zkServer.sh start
  # 启动Zookeeper
```

11. 【在node1、node2、node3上分别执行】 检查Zookeeper进程是否启动

```
1 jps
2
3 # 结果中找到有: QuorumPeerMain 进程即可
```

12. 【node1上操作】 验证Zookeeper

```
1 /export/server/zookeeper/zkCli.sh
2
3 # 进入到Zookeeper控制台中后, 执行
4 ls /
5
6 # 如无报错即配置成功
```

至此Zookeeper安装完成

Kafka集群安装部署

简介

Kafka是一款 **分布式的、去中心化的、高吞吐低延迟、订阅模式** 的消息队列系统。

同RabbitMQ一样，Kafka也是消息队列。不过RabbitMQ多用于后端系统，因其更加专注于消息的延迟和容错。

Kafka多用于大数据体系，因其更加专注于数据的吞吐能力。

Kafka多数都是运行在分布式（集群化）模式下，所以课程将以3台服务器，来完成Kafka集群的安装部署。

安装

1. 确保已经跟随前面的视频，安装并部署了JDK和Zookeeper服务

Kafka的运行依赖JDK环境和Zookeeper请确保已经有了JDK环境和Zookeeper

2. 【在node1操作】下载并上传Kafka的安装包

```
1 # 下载安装包
2 wget
  http://archive.apache.org/dist/kafka/2.4.1/kafka_2.1
  2-2.4.1.tgz
```

3. 【在node1操作】解压

```
1 mkdir -p /export/server          # 此文件夹如果不存在需先  
   创建  
2  
3 # 解压  
4 tar -zxvf kafka_2.12-2.4.1.tgz -C /export/server/  
5  
6 # 创建软链接  
7 ln -s /export/server/kafka_2.12-2.4.1  
   /export/server/kafka
```

4. 【在node1操作】修改Kafka目录内的config目录内的 **server.properties** 文件

```
1 cd /export/server/kafka/config  
2 # 指定broker的id  
3 broker.id=1  
4 # 指定 kafka的绑定监听的地址  
5 listeners=PLAINTEXT://node1:9092  
6 # 指定Kafka数据的位置  
7 log.dirs=/export/server/kafka/data  
8 # 指定Zookeeper的三个节点  
9 zookeeper.connect=node1:2181,node2:2181,node3:2181
```

5. 【在node1操作】将node1的kafka复制到node2和node3

```
1 cd /export/server  
2  
3 # 复制到node2同名文件夹  
4 scp -r kafka_2.12-2.4.1 node2:`pwd`/  
5 # 复制到node3同名文件夹  
6 scp -r kafka_2.12-2.4.1 node3:$PWD
```

6. 【在node2操作】

```
1 # 创建软链接
2 ln -s /export/server/kafka_2.12-2.4.1
  /export/server/kafka
3
4 cd /export/server/kafka/config
5 # 指定broker的id
6 broker.id=2
7 # 指定 kafka的绑定监听的地址
8 listeners=PLAINTEXT://node2:9092
9 # 指定Kafka数据的位置
10 log.dirs=/export/server/kafka/data
11 # 指定Zookeeper的三个节点
12 zookeeper.connect=node1:2181,node2:2181,node3:2181
```

7. 【在node3操作】

```
1 # 创建软链接
2 ln -s /export/server/kafka_2.12-2.4.1
  /export/server/kafka
3
4 cd /export/server/kafka/config
5 # 指定broker的id
6 broker.id=3
7 # 指定 kafka的绑定监听的地址
8 listeners=PLAINTEXT://node3:9092
9 # 指定Kafka数据的位置
10 log.dirs=/export/server/kafka/data
11 # 指定Zookeeper的三个节点
12 zookeeper.connect=node1:2181,node2:2181,node3:2181
```

8. 启动kafka

```
1 # 请先确保Zookeeper已经启动了
2
3 # 方式1: 【前台启动】分别在node1、2、3上执行如下语句
4 /export/server/kafka/bin/kafka-server-start.sh
  /export/server/kafka/config/server.properties
5
6 # 方式2: 【后台启动】分别在node1、2、3上执行如下语句
7 nohup /export/server/kafka/bin/kafka-server-start.sh
  /export/server/kafka/config/server.properties 2>&1
  >> /export/server/kafka/kafka-server.log &
```

9. 验证Kafka启动

```
1 # 在每一台服务器执行
2 jps
```



```
[root@node1 server]# jps
12930 NodeManager
14082 QuorumPeerMain
11843 DataNode
11652 NameNode
14340 HMaster
14599 ThriftServer
13705 JobHistoryServer
12746 ResourceManager
12237 SecondaryNameNode
19247 Jps
13428 WebAppProxyServer
14460 HRegionServer
14783 Kafka
```


测试Kafka能否正常使用

1. 创建测试主题

```
1 # 在node1执行, 创建一个主题
2 /export/server/kafka_2.12-2.4.1/bin/kafka-topics.sh --
  create --zookeeper node1:2181 --replication-factor 1 --
  partitions 3 --topic test
```

2. 运行测试, 请在FinalShell中打开2个node1的终端页面

```
1 # 打开一个终端页面, 启动一个模拟的数据生产者
2 /export/server/kafka_2.12-2.4.1/bin/kafka-console-
  producer.sh --broker-list node1:9092 --topic test
3 # 再打开一个新的终端页面, 在启动一个模拟的数据消费者
4 /export/server/kafka_2.12-2.4.1/bin/kafka-console-
  consumer.sh --bootstrap-server node1:9092 --topic test -
  -from-beginning
```

大数据集群（Hadoop生态）安装部署

简介

- 1) Hadoop是一个由Apache基金会所开发的分布式系统基础架构。
- 2) 主要解决, 海量数据的存储和海量数据的分析计算问题。

Hadoop HDFS 提供分布式海量数据存储能力

Hadoop YARN 提供分布式集群资源管理能力

Hadoop MapReduce 提供分布式海量数据计算能力

前置要求

- 请确保完成了集群化环境前置准备章节的内容
- 即：JDK、SSH免密、关闭防火墙、配置主机名映射等前置操作

Hadoop集群角色

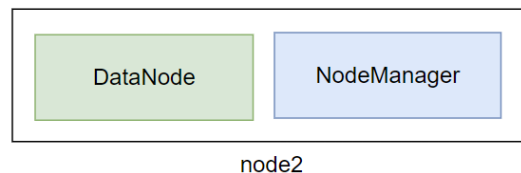
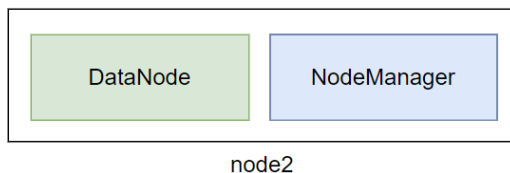
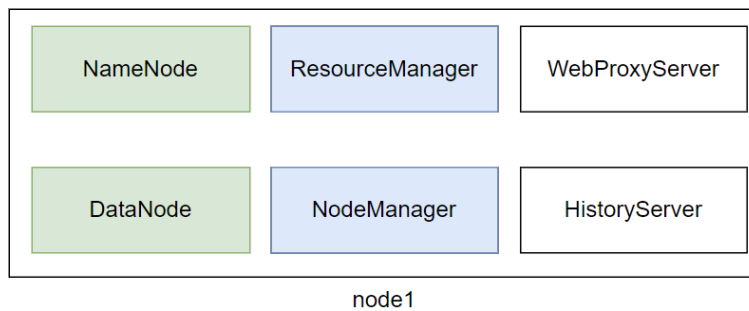
Hadoop生态体系中总共会出现如下进程角色：

1. Hadoop HDFS的管理角色：Namenode进程（**仅需1个即可（管理者一个就够）**）
2. Hadoop HDFS的工作角色：Datanode进程（**需要多个（工人，越多越好，一个机器启动一个）**）
3. Hadoop YARN的管理角色：ResourceManager进程（**仅需1个即可（管理者一个就够）**）
4. Hadoop YARN的工作角色：NodeManager进程（**需要多个（工人，越多越好，一个机器启动一个）**）
5. Hadoop 历史记录服务器角色：HistoryServer进程（**仅需1个即可（功能进程无需太多1个足够）**）
6. Hadoop 代理服务器角色：WebProxyServer进程（**仅需1个即可（功能进程无需太多1个足够）**）
7. Zookeeper的进程：QuorumPeerMain进程（**仅需1个即可（Zookeeper的工
作者，越多越好）**）

角色和节点分配

角色分配如下：

1. node1:Namenode、Datanode、ResourceManager、NodeManager、HistoryServer、WebProxyServer、QuorumPeerMain
2. node2:Datanode、NodeManager、QuorumPeerMain
3. node3:Datanode、NodeManager、QuorumPeerMain



安装

调整虚拟机内存

如上图，可以看出node1承载了太多的压力。同时node2和node3也同时运行了不少程序

为了确保集群的稳定，需要对虚拟机进行内存设置。

请在VMware中，对：

1. node1设置4GB或以上内存
2. node2和node3设置2GB或以上内存

大数据的软件本身就是集群化（一堆服务器）一起运行的。

现在我们在—台电脑中以多台虚拟机来模拟集群，确实会有很大的内存压力哦。

Zookeeper集群部署

略

Hadoop集群部署

1. 下载Hadoop安装包、解压、配置软链接

```
1 # 1. 下载
2 wget
  http://archive.apache.org/dist/hadoop/common/hadoop-
  3.3.0/hadoop-3.3.0.tar.gz
3
4 # 2. 解压
5 # 请确保目录/export/server存在
6 tar -zxvf hadoop-3.3.0.tar.gz -C /export/server/
7
8 # 3. 构建软链接
9 ln -s /export/server/hadoop-3.3.0
  /export/server/hadoop
```

2. 修改配置文件: `hadoop-env.sh`

Hadoop的配置文件要修改的地方很多, 请细心

cd 进入到/export/server/hadoop/etc/hadoop, 文件夹中, 配置文件都在这里

修改hadoop-env.sh文件

此文件是配置一些Hadoop用到的环境变量

这些是临时变量, 在Hadoop运行时有用

如果要永久生效, 需要写到/etc/profile中

```
1 # 在文件开头加入:
2 # 配置Java安装路径
3 export JAVA_HOME=/export/server/jdk
4 # 配置Hadoop安装路径
5 export HADOOP_HOME=/export/server/hadoop
```

```
6 # Hadoop hdfs配置文件路径
7 export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
8 # Hadoop YARN配置文件路径
9 export YARN_CONF_DIR=$HADOOP_HOME/etc/hadoop
10 # Hadoop YARN 日志文件夹
11 export YARN_LOG_DIR=$HADOOP_HOME/logs/yarn
12 # Hadoop hdfs 日志文件夹
13 export HADOOP_LOG_DIR=$HADOOP_HOME/logs/hdfs
14
15 # Hadoop的使用启动用户配置
16 export HDFS_NAMENODE_USER=root
17 export HDFS_DATANODE_USER=root
18 export HDFS_SECONDARYNAMENODE_USER=root
19 export YARN_RESOURCEMANAGER_USER=root
20 export YARN_NODEMANAGER_USER=root
21 export YARN_PROXYSERVER_USER=root
```

3. 修改配置文件: `core-site.xml`

如下, 清空文件, 填入如下内容

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?xml-stylesheet type="text/xsl"
  href="configuration.xsl"?>
3 <!--
4   Licensed under the Apache License, Version 2.0
  (the "License");
5   you may not use this file except in compliance
  with the License.
6   You may obtain a copy of the License at
7
8     http://www.apache.org/licenses/LICENSE-2.0
9
10  Unless required by applicable law or agreed to in
  writing, software
11  distributed under the License is distributed on
  an "AS IS" BASIS,
12  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND,
  either express or implied.
13  See the License for the specific language
  governing permissions and
```

```

14 limitations under the License. See accompanying
    LICENSE file.
15 →
16
17 <!-- Put site-specific property overrides in this
    file. →
18 <configuration>
19   <property>
20     <name>fs.defaultFS</name>
21     <value>hdfs://node1:8020</value>
22     <description></description>
23   </property>
24
25   <property>
26     <name>io.file.buffer.size</name>
27     <value>131072</value>
28     <description></description>
29   </property>
30 </configuration>

```

4. 配置: `hdfs-site.xml` 文件

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <?xml-stylesheet type="text/xsl"
    href="configuration.xsl"?>
3 <!--
4   Licensed under the Apache License, Version 2.0
    (the "License");
5   you may not use this file except in compliance
    with the License.
6   You may obtain a copy of the License at
7
8     http://www.apache.org/licenses/LICENSE-2.0
9
10  Unless required by applicable law or agreed to in
    writing, software
11  distributed under the License is distributed on
    an "AS IS" BASIS,
12  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND,
    either express or implied.

```

```
13   See the License for the specific language
14   governing permissions and
15   limitations under the License. See accompanying
16   LICENSE file.
17   →
18
19   <configuration>
20     <property>
21       <name>dfs.datanode.data.dir.perm</name>
22       <value>700</value>
23     </property>
24
25     <property>
26       <name>dfs.namenode.name.dir</name>
27       <value>/data/nn</value>
28       <description>Path on the local filesystem where
29       the NameNode stores the namespace and transactions
30       logs persistently.</description>
31     </property>
32
33     <property>
34       <name>dfs.namenode.hosts</name>
35       <value>node1,node2,node3</value>
36       <description>List of permitted DataNodes.
37     </description>
38   </property>
39
40   <property>
41     <name>dfs.blocksize</name>
42     <value>268435456</value>
43     <description></description>
44   </property>
45
46   <property>
47     <name>dfs.namenode.handler.count</name>
48     <value>100</value>
```

```
47     <description></description>
48 </property>
49
50 <property>
51     <name>dfs.datanode.data.dir</name>
52     <value>/data/dn</value>
53 </property>
54 </configuration>
```

5. 配置: `mapred-env.sh` 文件

```
1 # 在文件的开头加入如下环境变量设置
2 export JAVA_HOME=/export/server/jdk
3 export HADOOP_JOB_HISTORYSERVER_HEAPSIZE=1000
4 export HADOOP_MAPRED_ROOT_LOGGER=INFO,RFA
```

6. 配置: `mapred-site.xml` 文件

```
1 <?xml version="1.0"?>
2 <?xml-stylesheet type="text/xsl"
  href="configuration.xsl"?>
3 <!--
4   Licensed under the Apache License, Version 2.0
  (the "License");
5   you may not use this file except in compliance
  with the License.
6   You may obtain a copy of the License at
7
8       http://www.apache.org/licenses/LICENSE-2.0
9
10  Unless required by applicable law or agreed to in
  writing, software
11  distributed under the License is distributed on
  an "AS IS" BASIS,
12  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND,
  either express or implied.
13  See the License for the specific language
  governing permissions and
14  limitations under the License. See accompanying
  LICENSE file.
15 -->
```



```
16
17 <!-- Put site-specific property overrides in this
    file. -->
18
19 <configuration>
20   <property>
21     <name>mapreduce.framework.name</name>
22     <value>yarn</value>
23     <description></description>
24   </property>
25
26   <property>
27     <name>mapreduce.jobhistory.address</name>
28     <value>node1:10020</value>
29     <description></description>
30   </property>
31
32
33   <property>
34
35     <name>mapreduce.jobhistory.webapp.address</name>
36     <value>node1:19888</value>
37     <description></description>
38   </property>
39
40   <property>
41     <name>mapreduce.jobhistory.intermediate-done-
    dir</name>
42     <value>/data/mr-history/tmp</value>
43     <description></description>
44   </property>
45
46
47   <property>
48     <name>mapreduce.jobhistory.done-dir</name>
49     <value>/data/mr-history/done</value>
50     <description></description>
51   </property>
52 </property>
```

```
53     <name>yarn.app.mapreduce.am.env</name>
54     <value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
55 </property>
56 <property>
57     <name>mapreduce.map.env</name>
58     <value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
59 </property>
60 <property>
61     <name>mapreduce.reduce.env</name>
62     <value>HADOOP_MAPRED_HOME=$HADOOP_HOME</value>
63 </property>
64 </configuration>
```

7. 配置: `yarn-env.sh` 文件

```
1  # 在文件的开头加入如下环境变量设置
2  export JAVA_HOME=/export/server/jdk
3  export HADOOP_HOME=/export/server/hadoop
4  export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
5  export YARN_CONF_DIR=$HADOOP_HOME/etc/hadoop
6  export YARN_LOG_DIR=$HADOOP_HOME/logs/yarn
7  export HADOOP_LOG_DIR=$HADOOP_HOME/logs/hdfs
```

8. 配置: `yarn-site.xml` 文件

```
1  <?xml version="1.0"?>
2  <!--
3      Licensed under the Apache License, Version 2.0
4      (the "License");
5      you may not use this file except in compliance
6      with the License.
7      You may obtain a copy of the License at
8
9      http://www.apache.org/licenses/LICENSE-2.0
10
11     Unless required by applicable law or agreed to in
12     writing, software
13     distributed under the License is distributed on
14     an "AS IS" BASIS,
15     WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND,
16     either express or implied.
```

```
12   See the License for the specific language
13   governing permissions and
14   limitations under the License. See accompanying
15   LICENSE file.
16   →
17   <configuration>
18   <!-- Site specific YARN configuration properties -->
19   <property>
20       <name>yarn.log.server.url</name>
21       <value>http://node1:19888/jobhistory/logs</value>
22       <description></description>
23   </property>
24   <property>
25       <name>yarn.web-proxy.address</name>
26       <value>node1:8089</value>
27       <description>proxy server hostname and
28       port</description>
29   </property>
30
31   <property>
32       <name>yarn.log-aggregation-enable</name>
33       <value>true</value>
34       <description>Configuration to enable or disable
35       log aggregation</description>
36   </property>
37   <property>
38       <name>yarn.nodemanager.remote-app-log-
39       dir</name>
40       <value>/tmp/logs</value>
41       <description>Configuration to enable or disable
42       log aggregation</description>
43   </property>
```

```
44 <!-- Site specific YARN configuration properties --
45 >
46 <property>
47   <name>yarn.resourcemanager.hostname</name>
48   <value>node1</value>
49   <description></description>
50 </property>
51 <property>
52   <name>yarn.resourcemanager.scheduler.class</name>
53   <value>org.apache.hadoop.yarn.server.resourcemanager.scheduler.fair.FairScheduler</value>
54   <description></description>
55 </property>
56 <property>
57   <name>yarn.nodemanager.local-dirs</name>
58   <value>/data/nm-local</value>
59   <description>Comma-separated list of paths on
60   the local filesystem where intermediate data is
   written.</description>
61 </property>
62 <property>
63   <name>yarn.nodemanager.log-dirs</name>
64   <value>/data/nm-log</value>
65   <description>Comma-separated list of paths on
66   the local filesystem where logs are written.
   </description>
67 </property>
68 <property>
69   <name>yarn.nodemanager.log.retain-
70   seconds</name>
71   <value>10800</value>
```

```
74     <description>Default time (in seconds) to
      retain log files on the NodeManager Only applicable
      if log-aggregation is disabled.</description>
75   </property>
76
77
78
79   <property>
80     <name>yarn.nodemanager.aux-services</name>
81     <value>mapreduce_shuffle</value>
82     <description>Shuffle service that needs to be
      set for Map Reduce applications.</description>
83   </property>
84 </configuration>
```

9. 修改workers文件

```
1  # 全部内容如下
2  node1
3  node2
4  node3
```

10. 分发hadoop到其它机器

```
1  # 在node1执行
2  cd /export/server
3
4  scp -r hadoop-3.3.0 node2:`pwd`/
5  scp -r hadoop-3.3.0 node2:`pwd`/
```

11. 在node2、node3执行

```
1  # 创建软链接
2  ln -s /export/server/hadoop-3.3.0
    /export/server/hadoop
```

12. 创建所需目录

- 在node1执行:

```
1 mkdir -p /data/nn
2 mkdir -p /data/dn
3 mkdir -p /data/nm-log
4 mkdir -p /data/nm-local
```

◦ 在node2执行:

```
1 mkdir -p /data/dn
2 mkdir -p /data/nm-log
3 mkdir -p /data/nm-local
```

◦ 在node3执行:

```
1 mkdir -p /data/dn
2 mkdir -p /data/nm-log
3 mkdir -p /data/nm-local
```

13. 配置环境变量

在node1、node2、node3修改/etc/profile

```
1 export HADOOP_HOME=/export/server/hadoop
2 export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

执行 `source /etc/profile` 生效

14. 格式化NameNode, 在node1执行

```
1 hadoop namenode -format
```

hadoop这个命令来自于: `$HADOOP_HOME/bin`中的程序
由于配置了环境变量PATH, 所以可以在任意位置执行hadoop命令哦

15. 启动hadoop的hdfs集群, 在node1执行即可

```
1 start-dfs.sh
2
3 # 如需停止可以执行
4 stop-dfs.sh
```

start-dfs.sh这个命令来自于: `$HADOOP_HOME/sbin`中的程序

由于配置了环境变量PATH，所以可以在任意位置执行start-dfs.sh命令哦

16. 启动hadoop的yarn集群，在node1执行即可

```
1 start-yarn.sh
2
3 # 如需停止可以执行
4 stop-yarn.sh
```

17. 启动历史服务器

```
1 mapred --daemon start historyserver
2
3 # 如需停止将start更换为stop
```

18. 启动web代理服务器

```
1 yarn-daemon.sh start proxyserver
2
3 # 如需停止将start更换为stop
```

验证Hadoop集群运行情况

1. 在node1、node2、node3上通过jps验证进程是否都启动成功
2. 验证HDFS，浏览器打开：<http://node1:9870>

创建文件test.txt，随意填入内容，并执行：

```
1 hadoop fs -put test.txt /test.txt
2
3 hadoop fs -cat /test.txt
```

3. 验证YARN，浏览器打开：<http://node1:8088>

执行：

```
1 # 创建文件words.txt, 填入如下内容
2 itheima itcast hadoop
3 itheima hadoop hadoop
4 itheima itcast
5
6 # 将文件上传到HDFS中
7 hadoop fs -put words.txt /words.txt
8
9 # 执行如下命令验证YARN是否正常
10 hadoop jar
   /export/server/hadoop/share/hadoop/mapreduce/hadoop
   -mapreduce-examples-3.3.0.jar wordcount -
   Dmapred.job.queue.name=root.root /words.txt /output
```

大数据NoSQL数据库HBase集群部署

简介

HBase 是一种**分布式**、可扩展、支持海量数据存储的 NoSQL 数据库。

和Redis一样，HBase是一款Key-Value型存储的数据库。

不过和Redis设计方向不同

- Redis设计为少量数据，超快检索
- HBase设计为海量数据，快速检索

HBase在大数据领域应用十分广泛，现在我们来在node1、node2、node3上部署HBase集群。

安装

1. HBase依赖Zookeeper、JDK、Hadoop (HDFS) , 请确保已经完成前面
 - 集群化软件前置准备 (JDK)

- Zookeeper
- Hadoop
- 这些环节的软件安装

2. 【node1执行】下载HBase安装包

```
1 # 下载
2 wget
  http://archive.apache.org/dist/hbase/2.1.0/hbase-
  2.1.0-bin.tar.gz
3
4 # 解压
5 tar -zxvf hbase-2.1.0-bin.tar.gz -C /export/server
6
7 # 配置软链接
8 ln -s /export/server/hbase-2.1.0
  /export/server/hbase
```

3. 【node1执行】，修改配置文件，修改conf/hbase-env.sh文件

```
1 # 在28行配置JAVA_HOME
2 export JAVA_HOME=/export/server/jdk
3 # 在126行配置：
4 # 意思表示，不使用HBase自带的Zookeeper，而是用独立
  Zookeeper
5 export HBASE_MANAGES_ZK=false
6 # 在任意行，比如26行，添加如下内容：
7 export HBASE_DISABLE_HADOOP_CLASSPATH_LOOKUP="true"
```

4. 【node1执行】，修改配置文件，修改conf/hbase-site.xml文件

```
1 # 将文件的全部内容替换成如下内容：
2 <configuration>
3     <!-- HBase数据在HDFS中的存放的路径 -->
4     <property>
5         <name>hbase.rootdir</name>
6         <value>hdfs://node1:8020/hbase</value>
7     </property>
8     <!-- Hbase的运行模式。false是单机模式，true是分
  布式模式。若为false,Hbase和Zookeeper会运行在同一个JVM里面
  -->
```

```

 9      <property>
10          <name>hbase.cluster.distributed</name>
11          <value>true</value>
12      </property>
13      <!-- ZooKeeper的地址 -->
14      <property>
15          <name>hbase.zookeeper.quorum</name>
16          <value>node1,node2,node3</value>
17      </property>
18      <!-- ZooKeeper快照的存储位置 -->
19      <property>
20
21          <name>hbase.zookeeper.property.dataDir</name>
22          <value>/export/server/apache-zookeeper-
23          3.6.0-bin/data</value>
24      </property>
25      <!-- V2.1版本, 在分布式情况下, 设置为false -->
26      <property>
27          <name>hbase.unsafe.stream.capability.enforce</name>
28          <value>false</value>
29      </property>
30  </configuration>

```

5. 【node1执行】，修改配置文件，修改 **conf/regionservers** 文件

```

1  # 填入如下内容
2  node1
3  node2
4  node3

```

6. 【node1执行】，分发hbase到其它机器

```

1  scp -r /export/server/hbase-2.1.0
   node2:/export/server/
2  scp -r /export/server/hbase-2.1.0
   node3:/export/server/

```

7. 【node2、node3执行】，配置软链接

```
1 ln -s /export/server/hbase-2.1.0  
  /export/server/hbase
```

8. 【node1、node2、node3执行】，配置环境变量

```
1 # 配置在/etc/profile内，追加如下两行  
2 export HBASE_HOME=/export/server/hbase  
3 export PATH=$HBASE_HOME/bin:$PATH  
4  
5 source /etc/profile
```

9. 【node1执行】启动HBase

请确保：Hadoop HDFS、Zookeeper是已经启动了的

```
1 start-hbase.sh  
2  
3 # 如需停止可使用  
4 stop-hbase.sh
```

由于我们配置了环境变量`export PATH=$PATH:$HBASE_HOME/bin`
`start-hbase.sh`即在`$HBASE_HOME/bin`内，所以可以无论当前目录在
哪，均可直接执行

10. 验证HBase

浏览器打开：<http://node1:16010>，即可看到HBase的WEB UI页面

11. 简单测试使用HBase

【node1执行】

```
1 hbase shell
2
3 # 创建表
4 create 'test', 'cf'
5
6 # 插入数据
7 put 'test', 'rk001', 'cf:info', 'itheima'
8
9 # 查询数据
10 get 'test', 'rk001'
11
12 # 扫描表数据
13 scan 'test'
```

分布式内存计算Spark环境部署

注意

本小节的操作，基于：[大数据集群（Hadoop生态）安装部署](#)环节中所构建的Hadoop集群

如果没有Hadoop集群，请参阅前置内容，部署好环境。

简介

Spark是一款分布式内存计算引擎，可以支撑海量数据的分布式计算。

Spark在大数据体系是明星产品，作为最新一代的综合计算引擎，支持离线计算和实时计算。

在大数据领域广泛应用，是目前世界上使用最多的大数据分布式计算引擎。

我们将基于前面构建的Hadoop集群，部署Spark Standalone集群。

安装

1. 【node1执行】下载并解压

```
1 wget https://archive.apache.org/dist/spark/spark-2.4.5/spark-2.4.5-bin-hadoop2.7.tgz
2
3 # 解压
4 tar -zxvf spark-2.4.5-bin-hadoop2.7.tgz -C /export/server/
5
6 # 软链接
7 ln -s /export/server/spark-2.4.5-bin-hadoop2.7 /export/server/spark
```

2. 【node1执行】修改配置文件名称

```
1 # 改名
2 cd /export/server/spark/conf
3 mv spark-env.sh.template spark-env.sh
4 mv slaves.template slaves
```

3. 【node1执行】修改配置文件， `spark-env.sh`

```
1 ## 设置JAVA安装目录
2 JAVA_HOME=/export/server/jdk
3
4 ## HADOOP软件配置文件目录，读取HDFS上文件和运行YARN集群
5 HADOOP_CONF_DIR=/export/server/hadoop/etc/hadoop
6 YARN_CONF_DIR=/export/server/hadoop/etc/hadoop
7
8 ## 指定spark老大Master的IP和提交任务的通信端口
9 export SPARK_MASTER_HOST=node1
10 export SPARK_MASTER_PORT=7077
11
12 SPARK_MASTER_WEBUI_PORT=8080
13 SPARK_WORKER_CORES=1
```

```
14 | SPARK_WORKER_MEMORY=1g
```

4. 【node1执行】修改配置文件, **slaves**

```
1 | node1
2 | node2
3 | node3
```

5. 【node1执行】分发

```
1 | scp -r spark-2.4.5-bin-hadoop2.7 node2:$PWD
2 | scp -r spark-2.4.5-bin-hadoop2.7 node3:$PWD
```

6. 【node2、node3执行】设置软链接

```
1 | ln -s /export/server/spark-2.4.5-bin-hadoop2.7
   | /export/server/spark
```

7. 【node1执行】启动Spark集群

```
1 | /export/server/spark/sbin/start-all.sh
2 |
3 | # 如需停止, 可以
4 | /export/server/spark/sbin/stop-all.sh
```

8. 打开Spark监控页面, 浏览器打开: **http://node1:8081**

9. 【node1执行】提交测试任务

```
1 | /export/server/spark/bin/spark-submit --master
   | spark://node1:7077 --class
   | org.apache.spark.examples.SparkPi
   | /export/server/spark/examples/jars/spark-
   | examples_2.11-2.4.5.jar
```

分布式内存计算Flink环境部署

注意

本小节的操作，基于：[大数据集群（Hadoop生态）安装部署](#)环节中所构建的Hadoop集群

如果没有Hadoop集群，请参阅前置内容，部署好环境。

简介

Flink同Spark一样，是一款分布式内存计算引擎，可以支撑海量数据的分布式计算。

Flink在大数据体系同样是明星产品，作为最新一代的综合计算引擎，支持离线计算和实时计算。

在大数据领域广泛应用，是目前世界上除去Spark以外，应用最为广泛的分布式计算引擎。

我们将基于前面构建的Hadoop集群，部署Flink Standalone集群

Spark更加偏向于离线计算而Flink更加偏向于实时计算。

安装

1. 【node1操作】下载安装包

```
1 wget https://archive.apache.org/dist/flink/flink-
  1.10.0/flink-1.10.0-bin-scala_2.11.tgz
2
3 # 解压
4 tar -zxvf flink-1.10.0-bin-scala_2.11.tgz -C
  /export/server/
5
6 # 软链接
7 ln -s /export/server/flink-1.10.0
  /export/server/flink
```

2. 【node1操作】修改配置文件, `conf/flink-conf.yaml`

```
1 # jobManager 的IP地址
2 jobmanager.rpc.address: node1
3 # JobManager 的端口号
4 jobmanager.rpc.port: 6123
5 # JobManager JVM heap 内存大小
6 jobmanager.heap.size: 1024m
7 # TaskManager JVM heap 内存大小
8 taskmanager.heap.size: 1024m
9 # 每个 TaskManager 提供的任务 slots 数量大小
10 taskmanager.numberOfTaskSlots: 2
11 #是否进行预分配内存, 默认不进行预分配, 这样在我们不使用flink
  集群时候不会占用集群资源
12 taskmanager.memory.preallocate: false
13 # 程序默认并行计算的个数
14 parallelism.default: 1
15 #JobManager的Web界面的端口 (默认: 8081)
16 jobmanager.web.port: 8081
```

3. 【node1操作】, 修改配置文件, `conf/slaves`

```
1 node1
2 node2
3 node3
```

4. 【node1操作】分发Flink安装包到其它机器


```
1 | cd /export/server
2 | scp -r flink-1.10.0 node2:`pwd`/
3 | scp -r flink-1.10.0 node3:`pwd`/
```

5. 【node2、node3操作】

```
1 | # 配置软链接
2 | ln -s /export/server/flink-1.10.0
   /export/server/flink
```

6. 【node1操作】，启动Flink

```
1 | /export/server/flink/bin/start-cluster.sh
```

7. 验证Flink启动

```
1 | # 浏览器打开
2 | http://node1:8081
```

8. 提交测试任务

【node1执行】

```
1 | /export/server/flink/bin/flink run
   /export/server/flink-
   1.10.0/examples/batch/WordCount.jar
```

运维监控Zabbix部署

简介

Zabbix 由 Alexei Vladishev 创建，目前由其成立的公司—— Zabbix SIA 积极的持续开发更新维护， 并为用户提供技术支持服务。

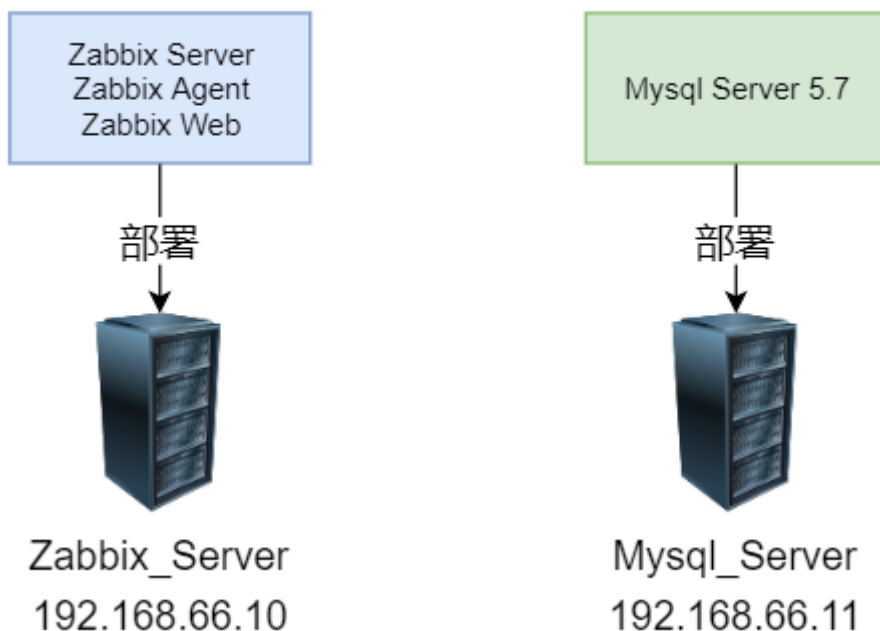
Zabbix 是一个 **企业级分布式开源监控解决方案**。

Zabbix 软件能够 **监控** 众多网络参数和服务器的 **健康度、完整性**。Zabbix 使用灵活的告警机制，允许用户为几乎任何事件配置基于邮件的告警。这样用户可以快速响应服务器问题。Zabbix 基于存储的数据提供出色的报表和数据可视化功能。这些功能使得 Zabbix 成为容量规划的理想选择。

安装

安装整体步骤：

1. 准备Linux 服务器(虚拟机)
2. 安装MySQL
3. 安装zabbix(包含 server agent web)
4. 配置 mysql, 为zabbix创建表结构
5. 配置zabbix server
6. 启动并开启开机自启动



安装前准备 - Mysql

安装ZabbixServer需要先安装好Mysql数据库

课程使用Mysql 5.7

安装步骤:

```
1 # 安装Mysql yum库
2 rpm -Uvh http://repo.mysql.com//mysql57-community-
  release-el7-7.noarch.rpm
3
4 # yum安装Mysql
5 yum -y install mysql-community-server
6
7 # 启动Mysql设置开机启动
8 systemctl start mysqld
9 systemctl enable mysqld
10
11 # 检查Mysql服务状态
12 systemctl status mysqld
13
14 # 第一次启动mysql, 会在日志文件中生成root用户的一个随机密码, 使
  用下面命令查看该密码
15 grep 'temporary password' /var/log/mysqld.log
16
17 # 修改root用户密码
18 mysql -u root -p -h localhost
19 Enter password:
20
21 mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY
  'Root!@#$';
22
23 # 如果你想设置简单密码, 需要降低Mysql的密码安全级别
24 set global validate_password_policy=LOW; # 密码安全级别低
25 set global validate_password_length=4;   # 密码长度最低4
  位即可
26
27 # 然后就可以用简单密码了 (课程中使用简单密码, 为了方便, 生产中不
  要这样)
28 ALTER USER 'root'@'localhost' IDENTIFIED BY 'root';
```

```
29 mysql> grant all privileges on *.* to root@'%'
identified by 'root';
```

安装Zabbix Server 和 Zabbix Agent

初始安装，我们先安装ZabbixServer以及在Server本机安装Agent。

打开官网下载页面：https://www.zabbix.com/download?zabbix=4.0&os_distribution=centos&os_version=7&db=mysql

1 Choose your platform

ZABBIX VERSION	OS DISTRIBUTION	OS VERSION	DATABASE
4.4	Red Hat Enterprise Linux	8	MySQL
4.2	CentOS	7	PostgreSQL
4.0 LTS	Oracle Linux	6	
3.0 LTS	Ubuntu		
	Debian		
	SUSE Linux Enterprise Server		
	Raspbian		

选择对应的版本，然后再下面官网给出了具体的安装命令，使用 **rpm** 和 **yum** 来进行安装。

需要有网络。

以下内容来自官方页面

a. 安装Zabbix yum库

documentation

```
1 rpm -Uvh
  https://repo.zabbix.com/zabbix/4.0/rhel/7/x86_64/zabbix-
  release-4.0-2.el7.noarch.rpm
2 yum clean all
```

b. 安装Zabbix Server、前端、Agent

```
1 yum -y install zabbix-server-mysql zabbix-web-mysql
  zabbix-agent
2 # 如果只需要安装Agent的话
3 yum -y install zabbix-agent
```

c. 初始化MySQL数据库

documentation

在MySQL中操作

```
1 # 登录MySQL 数据库
2 mysql -uroot -pYourPassword
3 mysql> create database zabbix character set utf8 collate
  utf8_bin;
4 mysql> grant all privileges on zabbix.* to
  zabbix@localhost identified by 'zabbix';
5 # 或者: grant all privileges on zabbix.* to zabbix@%'
  identified by 'zabbix';
6 mysql> quit;
```

测试在Zabbix Server服务器上能否远程登录MySQL, 如果可以登录继续向下走。

Import initial schema and data. You will be prompted to enter your newly created password.

```
1 # zcat /usr/share/doc/zabbix-server-mysql*/create.sql.gz
  | mysql -uzabbix -p zabbix
```

d. 为Zabbix Server配置数据库

Edit file /etc/zabbix/zabbix_server.conf

```
1 DBPassword=password
2 DBHost=mysql-host-ip-or-hostname
```

e. 配置Zabbix的PHP前端

Edit file `/etc/httpd/conf.d/zabbix.conf`, uncomment and set the right timezone for you. `# php_value date.timezone Asia/Shanghai`

Start Zabbix server and agent processes and make it start at system boot:

```
1 | systemctl restart zabbix-server zabbix-agent httpd # 启动、重启
2 | systemctl enable zabbix-server zabbix-agent httpd # 开机自启
```

Now your Zabbix server is up and running!

配置zabbix 前端 (WEB UI)

打开: <http://192.168.88.131/zabbix>

即可进入Zabbix页面, 在首次打开的时候, 会进入设置页面, 如图:



点击下一步, 会检查相应的设置是否都正常

Welcome

Check of pre-requisites

Configure DB connection

Zabbix server details

Pre-installation summary

Install

	Current value	Required	
PHP version	5.4.16	5.4.0	OK
PHP option "memory_limit"	128M	128M	OK
PHP option "post_max_size"	16M	16M	OK
PHP option "upload_max_filesize"	2M	2M	OK
PHP option "max_execution_time"	300	300	OK
PHP option "max_input_time"	300	300	OK
PHP option "date.timezone"	Asia/Shanghai		OK
PHP databases support	MySQL		OK
PHP bcmath	on		OK
PHP mbstring	on		OK
PHP option "mbstring.func_overload"	off	off	OK

Back

Next step

如果一切正常，点击下一步。

配置DB连接

Please create database manually, and set the configuration parameters for connection to this database. Press "Next step" button when done.

Welcome

Check of pre-requisites

Configure DB connection

Zabbix server details

Pre-installation summary

Install

Database type

MySQL

Database host

192.168.66.11

Database port

3306

0 - use default port

Database name

zabbix

User

zabbix

Password

Back

Next step

按具体情况填写即可

配置Server细节

ZABBIX

Welcome

Check of pre-requisites

Configure DB connection

Zabbix server details

Pre-installation summary

Install

Zabbix server details

Please enter the host name or host IP address and port number of the Zabbix server, as well as the name of the installation (optional).

Host

localhost

Port

10051

Name

itcast-test

具体配置即可，Name表示这个Zabbix服务的名字，这里起名叫ITHEIMA-TEST

安装前总结预览

检查确认没有问题就下一步

ZABBIX

Welcome

Check of pre-requisites

Configure DB connection

Zabbix server details

Pre-installation summary

Install

Pre-installation summary

Please check configuration parameters. If all is correct, press "Next step" button, or "Back" button to change configuration parameters.

Database type

MySQL

Database server

192.168.66.11

Database port

3306

Database name

zabbix

Database user

zabbix

Database password

Zabbix server

localhost

Zabbix server port

10051

Zabbix server name

ITHEIMA-TEST

Back

Next step

配置完成

ZABBIX

Welcome

Check of pre-requisites

Configure DB connection

Zabbix server details

Pre-installation summary

Install

Install

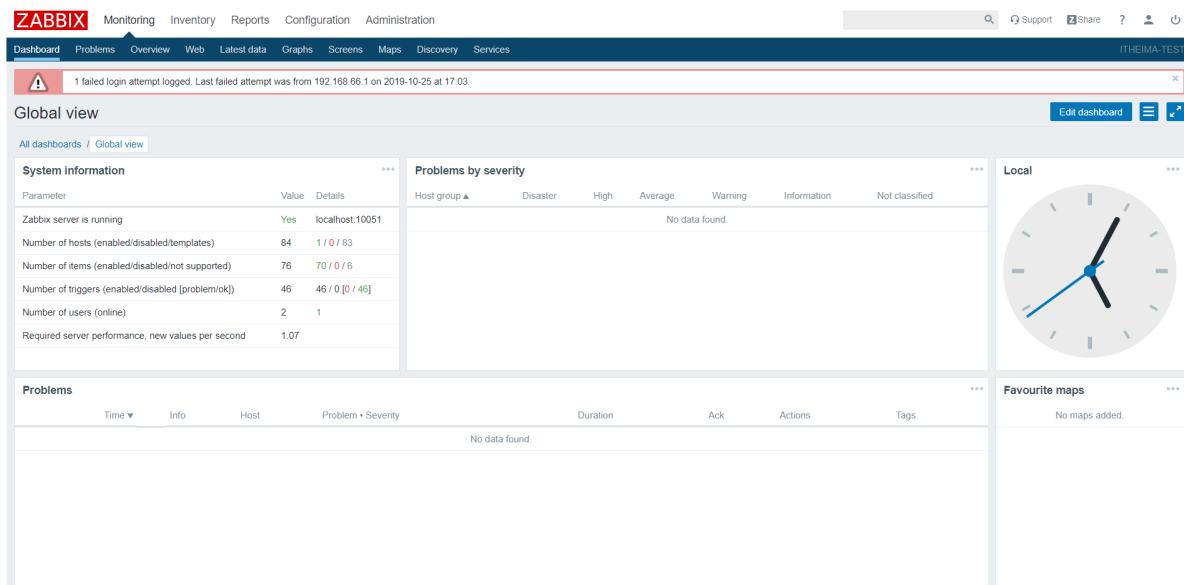
Congratulations! You have successfully installed Zabbix frontend.

Configuration file "/etc/zabbix/web/zabbix.conf.php" created.

初始管理员账户Admin密码zabbix

输入账户密码后，就能进入zabbix页面了。

如下图：



现在是一个崭新的zabbix等待我们去探索。

运维监控Grafana部署

简介

安装

部署形式

Grafana支持两种部署形式

1. 自行部署，可以部署在操作系统之上。自行提供服务器，域名等。
2. **Grafana**官方托管。无需安装，在线注册即可得到一个专属于自己的**Grafana**，但是要花钱的。是一种**SaaS**服务

我们课程选择方式1

安装

Grafana支持常见的绝大多数操作系统，如**windows mac linux** 同时也支持部署在**docker**中。

大多数情况下，**Grafana**都是部署在**linux**服务器之上。所以本课程也是基于**Linux**系统来讲解。

对**windows mac**系统 或 **docker**部署有兴趣的同学，请参考：<https://grafana.com/grafana/download>

我们部署**Grafana**可以使用**YUM**来进行部署。

```
1 # 创建一个文件
2 vim /etc/yum.repos.d/grafana.repo
3
4 # 将下面的内容复制进去
5 [grafana]
6 name=grafana
7 baseurl=https://packages.grafana.com/oss/rpm
8 repo_gpgcheck=1
9 enabled=1
10 gpgcheck=1
11 gpgkey=https://packages.grafana.com/gpg.key
12 sslverify=1
13 sslcacert=/etc/pki/tls/certs/ca-bundle.crt
14
15 # 最后安装
16 yum install grafana
```

配置说明

grafana-server具有许多配置选项，这些选项可以在**.ini**配置文件中指定，也可以使用环境变量指定。

Note. **Grafana** needs to be restarted for any configuration changes to take effect.

配置文件注释

; 符号在 **.ini** 文件中全局表示注释 ()

配置文件路径

如果是自己解压安装，或者自行编译的方式安装，配置文件在：

- 默认： **\$WORKING_DIR/conf/defaults.ini**
- 自定义： **\$WORKING_DIR/conf/custom.ini**
- 自定义配置文件路径可以被参数 **--config** 覆盖

对于 **YUM RPM** 安装的方式，配置文件在： **/etc/grafana/grafana.ini**

使用环境变量

可以使用以下语法使用环境变量来覆盖配置文件中的所有选项：

```
1 GF_<SectionName>_<KeyName>
```

其中 **SectionName** 是方括号内的文本。一切都应为大写，**.** 应替换为 **_** 例如，给定以下配置设置：

```
1 # default section
2 instance_name = ${HOSTNAME}
3
4 [security]
5 admin_user = admin
6
7 [auth.google]
8 client_secret = 0ldS3cretKey
```

Then you can override them using:

```
1 export GF_DEFAULT_INSTANCE_NAME=my-instance
2 export GF_SECURITY_ADMIN_USER=true # GF_ 固定 SECURITY
  是SectionName ADMIN_USER 是配置的关键字 转大写 . 转 _
3 export GF_AUTH_GOOGLE_CLIENT_SECRET=newS3cretKey
```

开始配置

Grafana支持使用Sqlite3 Postgresql Mysql这三种数据库作为其元数据的存储。

我们课程使用Mysql. 和zabbix的元数据mysql共用一个实例

只需要配置如下内容即可：

```
[database]
# You can configure the database
# as separate properties

# Either "mysql", "postgresql" or "sqlite3"
type = mysql
host = 127.0.0.1:3306
name = grafana
user = root
# If the password contains special characters, use "password"
password = root
```

并登陆mysql, 执行：

```
create database grafana CHARACTER SET utf8 COLLATE
utf8_general_ci;
```

创建Grafana使用的数据库作为元数据存储。

启动

```
1 systemctl daemon-reload
2 systemctl start grafana-server
3 systemctl enable grafana-server
```

浏览器打开: <http://node1:3000>

默认账户密码: admin/admin